The Sequences of Bundles of Line Segments for Autonomous Robots with Limited Vision Range to Escape from Blind Alley Regions

Phan Thanh An[1] and Pham Hoang Anh[2] and Tran Thanh Binh[2] and Tran Van Hoai[2]

[1] Institute of Mathematical and Computational Sciences (IMACS)
Ho Chi Minh City University of Technology (HCMUT)
268 Ly Thuong Kiet Street, District 10, Ho Chi Minh City, Vietnam
[2] Faculty of Computer Science and Engineering
Ho Chi Minh City University of Technology (HCMUT)
268 Ly Thuong Kiet Street, District 10, Ho Chi Minh City, Vietnam

ARTICLE HISTORY
Compiled June 29, 2025

ABSTRACT
Consider the following problem: A robot operating in a 2D environment with a limited vision range finds a path to a goal in an unknown environment containing obstacles. In this paper, we propose a novel algorithm to solve the problem. In some special cases, our algorithm is convergent with respect to $\|.\|$. The problem involves discovering the environmental map and blind alley regions, that are bounded by obstacles, and it provides no possible passage for robots except in and out of their path entry occur, the robot has to return back to some positions outside to escape from such regions such that the returned path is not longer than the path entry (Blind Alley Region problem, (BAR) problem, in short). To solve the (BAR) problem, sequences of bundles of line segments during the robot's traveling are constructed in our algorithm.

Some advantages of our algorithm are that a) It reduces search space in blind alley regions because it only works on the sequences of bundles of the line segments built by the robot's limited vision range. b) Our algorithm ensures that the returned path to escape from the regions is not longer than the previous path of the robot. c) Due to the construction of the sequences of bundles of line segments, our paths are not always "close" obstacles and the number of turns of such paths is smaller ones determined by other shortest path algorithms (e.g., A*, RRT*).

Our algorithm is implemented in Python and we experience the algorithm on some autonomous robots with different vision ranges in real environment. We also compare our result with RRTX, a state-of-art local path-planning algorithm, and A*, a basic one. The experimental results show that our algorithm provides better solutions than RRTX and A* results in some specific circumstances.

KEYWORDS
Astar algorithm and Blind alley region and Bundle of line segments and Polygonal obstacle and Rapidly-exploring random tree and Robot path planning and Shortest path

* Contact: thanhan@hcmut.edu.vn, hoai@hcmut.edu.vn

1. Introduction

Path planning is one of the most important requirements of autonomous robots. Path planning approaches can be divided into two categories: global and local ones. The global path planning method computes a path offline based on whole map information, whereas the local path planning requires local map information to move in an unknown or dynamic environment. Local path planning algorithms are usually used for obstacle avoidance and the change of environment (see [1] and [16]).

Local path planning is also due to the limit of robot's vision range, say r. A robot uses a so-called LiDar equipment for light detection and ranging (for RS-LiDar-32, $r = 200m$, for LiDar LDS-01, $r = 35m$, for LiDar RP LIDAR A2M6, $r = 18m$, for SLAMTEC RPLiDar A1, $r = 12m$, for SLAMTEC RPLiDar A2, $r = 16m$, for SLAMTEC RPLiDar A3, $r = 25m$, and for SLAMTEC RPLiDar S1, $r = 40m$), see [1] and [32]. At our institute[2], three our robots have different vision ranges (see Figure 1).

Figure 1.: The four wheeled robot (left) has $r = 1m$, the PuppyPi (center) has $r = 1m$, and the TurtleBot3 burger (right) has $r = 0.88m$.

However, the local path planning can not always produce optimal solutions and there arises a so-called blind alley region problem ((BAR) problem, in short):

(BAR) The robot is in a blind alley region, that is bounded by obstacles and it provides no possible passage for robots except in and out of its path entry, then the robot has to return to some position outside the region such that the returned path is not longer than the path entry.

See some blind alley regions given in Figure 2. Dead-end, bug-trap, and the U-shaped regions are special cases of blind alley regions.

Under the limit of robot's vision range, the full knowledge of the map and environment is not available. How can we solve (BAR) problem?

The methods of Tashtoush et al. [37], letting the robot escape the U-shaped obstacle by backtracking the initial path, Kang et al. [14] using genetic algorithm and

[2]Institute of Mathematical and Computational Sciences - IMACS, HCMUT `https://imacs.hcmut.edu.vn/`

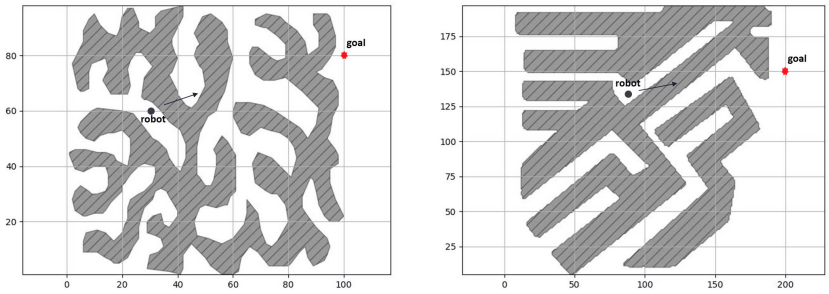


Figure 2.: Two blind alley regions: blind alley region 1 (left); Blind alley region 2 (right). Shaded regions are obstacles. The robots are in these regions.

online training, which takes time, to help the robot escape the dead-end, Multilayer Decision-Based Fuzzy Logic, proposed by Kamil and Moghrabiah [13], Feng et al. [8] using Local-and-Global Strategy are not already for solving (BAR) problem as their returned paths may be longer than the path entries.

In addition, the cross-fertilization of motion planning and machine learning solves planning problems. Among others, WPN - waypoint planning networks (see [38]) and MPNet - motion planning networks (see [30]), are used.

In this paper, we propose a novel algorithm for path planning problems in unknown static 2D environments. The robot uses a sensor (LiDar) to collect local information for environmental perception. The main contribution of this paper is to build a simple space during the robot's traveling: sequences of bundles of line segments. Then, to solve the (BAR) problem, the robot uses such sequences to escape from blind alley regions effectively (Section 4): To get a shorter path than the previous one, we find a shortest path that joins two locations along the sequence. In [18], we already show some geometrical properties of shortest paths joining two given points along the sequence. In this paper, we use Algorithm 7 [24] for finding approximately such paths (Section 3). Some advantages of our method are shown in Section 4.

The remaining of this paper is organized as follows. Literature work is presented in Section 2. Section 3 presents the approximate shortest path problem in the plane: find a shorter path then the polyline joining two locations along the sequence of bundles of line segments. Section 4 describes our proposed approach: build a simple space during the robot's traveling: sequences of bundles of line segments. Then the robot use it to escape blind alley regions effectively and (BAR) problem is solved (Section 4. Some special cases of our algorithm are given in Sect. 5. The experimental results that show that our algorithm provides better solutions than RRT* and A* results. Their implementations for the robot TurtleBot 3 Burger, for the wheeled robot, and for the PuppyPi Hiwonder Quadruped robot which have the vision ranges of 80mm, 100mm, and 100mm, respectively in a real enviroment are summarized in Section 6.

In the paper, there is no robot's model and robot's controller because the research is focused on algorithmic innovation rather than hardware implementation.

2. Related Work

2.1. The global path planning

Many researches have studied global techniques such as cell decomposition, road-maps, potential fields, search-based, and sampling-based methods (see [15] [20], [21], [28], [33], and [34]). Generally, these techniques assume that a complete model of the world is available. Therefore, they can calculate a path offline and usually archive an optimal solution (for cell decomposition method, see [33] and [34], for Road-map method, see [20], [21], for potential fields method, see [15]).

The following parts discuss more details about such methods.

Cell decomposition method: this method divides the free space of the robot's configuration into simple and small regions called cells. Then, from those cells, it builds an adjacency graph where vertices correspond to cells and edges connect adjacent cells. Base on the adjacency graph, it detects cells that contain the start point and goal point. Then, it searches on the adjacency graph for a path. Some implementations based on this method could be found in [33] and [34].

Road-map method: from the robot's free space configuration, it builds a road-map inside that space. Then, it finds a map. Similarly, it finds another path from the goal point to a point p^g in the road-map. After that, it connects a road from p^s to p^g to get a path from start to goal. Two well-known road-map methods that achieved good results with various roads are the visibility graph and Voronoi diagram. Some works using this method could be found in [20] and [21].

Potential fields method: this method base on the idea of the attractive and repulsive field. If two things are attractive, they head each other. The goal point is attractive to a robot. On the other hand, if two things are repulsive, they detach each other. Base on the above principle, it uses global map information to builds a potential field where the robot can head to the goal and avoid obstacles. Implementation using this method can be found in [15].

Search-based method: this method finds a path between two locations in an unknown, partially known, or known environment. It uses blind search such as Breadth-first search (known as wave front) or Depth-first search for the unknown or partially known environment. It is not an effective method. In terms of the known environment, it used an evaluation function for searching. Thus, it is more efficient. Some well-known search algorithms are Dijkstra, A^*, D^*, D^* lite, DD^* lite.

Among the others, the sampling-based method is used to reduce search space. Sampling-based method: search-based method is computation expensive if configuration space is enormous. Therefore, to reduce search space, it proposed a sampling method. The sampling approach does not expand all nearby space, instead rapidly exploring random trees (RRT) are used (see [17] and [31]). It is computational efficiency and effectiveness and its ability to find a feasible motion plan relatively quickly, even in high-dimensional space. Some researches applying RRT family algorithms for path planning can be found in [17] and [31].

The following subsection discusses local path planning that deals with obstacle avoidance or unknown environments in which all the above-mentioned methods do not work. For the sake of simplicity, the obstacles are supposed to be polygons.

2.2. The local path planning

The main functionality of the local path planning is to adapt an initial path, which is usually calculated by a global path planning, to an unforeseen situation or change of environment.

The following parts discuss more details about this catalogue.

Well-known research for real-time and high-speed obstacle avoidance in populated and dynamic environments is the dynamic window approach by [9] and Bayesian optimization sampling, and deep reinforcement learning models by [8]. This research collects local information (window) then controls the robot base on the motion dynamics of the robot. Another the state-of-art local path-planning is RRTX by Michael et al. [31]. RRTX is an asymptotically optimal sampling-based re-planning for dynamic environments where obstacles unpredictably appear, disappear, or move.

Initially, RRTx build an adjacency graph integrated with RRT*-based tree. Then, whenever obstacles appear, the graph rewiring cascade quickly the existing search-graph and reroutes its shortest-path-to-goal RRTx to reflect the new local information.

While the global path is usually fixed, some local path planning algorithms are proposed to make such path smoother by set some knots (control point in the global path). Clothoid curves transit to a curve from linear change path at local. Polynomial curves are commonly implemented to meet the constraints needed in the points of devices or environment. Bézier Curves generate parametric curves that rely on control points to define their shape (i.e., turns, roundabouts, lane change, obstacle avoidance). Spline Curves: a spline is a piece-wise polynomial parametric curve divided into sub-intervals that can be defined as polynomial curves, b-splines, or clothoid curves to avoid dynamic obstacles. Bug Family Algorithm: almost all global path planning algorithms assume global knowledge as input (see [29]). This family algorithm just needs local knowledge of the environment and a goal. The idea of this method is base on bug behaviors that follow a wall and move toward the given goal. This approach work in a very simple world, if there are many obstacles these algorithms do not work well. Among the others are bug family algorithms (that can be found in [11],[12], and [27]).

3. The Approximate Shortest Path Problem

During operations of the robot, suitable sequences of bundles of line segments of the explored map are constructed for finding the shortest paths between two locations on (this can be seen later in Sect. 4).

3.1. Sequences of bundles of line segments

For any points x, y in the plane, we denote $\|x - y\|$ is the length of the line segment xy. Let $\{ce_1, ce_2, \dots ce_n\}(n \geq 1)$ be a sequence of n distinct line segments in the plane in the clockwise or counterclockwise order such that $ce_2, \dots ce_{n-1}$ be between ce_1 and ce_n. The sequence is called a *bundle of n line segments* with the *vertex* c A point is considered as a degenerate bundle of line segments. Let us denote $B(c, r) = \{p \in \mathbb{R}^2 | \, \|p - c\| < r\}$, $\overline{B}(c, r) = \{p \in \mathbb{R}^2 | \, \|p - c\| \leq r\}$, and $\mathrm{bd}\overline{B}(c, r) = \{p \in \mathbb{R}^2 | \, \|p - c\| = r\}$, where $r > 0$.

An ordered set of bundles at vertices $c_1, c_2, \dots, c_N$ is called a *sequence* $c_1, c_2, \dots, c_N$ *of bundles of line segments*. Assume that line segments of bundles of disjoint vertices

do not intersect each other.

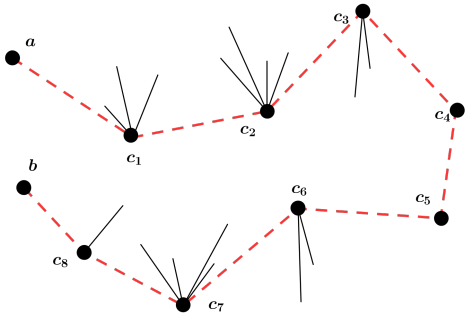


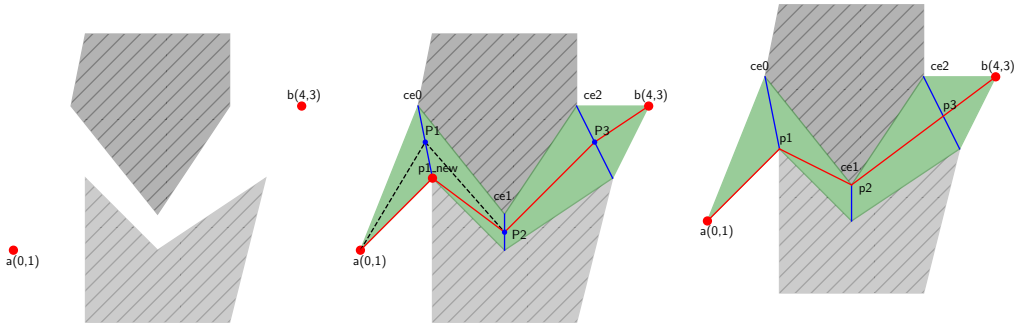
Figure 3.: A sequence of bundles of line segments and the polyline formed a, the vertices, and b.

Given a sequence $c_1, c_2, \ldots, c_n$ of bundles of line segments and two points a and b. Let τ be a polyline joining $a, c_1, c_2, \ldots, c_n$ and b. We adopt the convention that a non-degenerate bundle c_i consists of one or more line segments whose endpoint(s) is/are on the same side of or on τ.

Definition 3.1. The *angle of* τ *at the vertex* c_i of the above bundle is the angle at center with respect to the sector of $\overline{B}(c_i, r)$ which contains the bundle with a large enough radius r, for $1 \leq i \leq n$.

If the angle of τ at the vertex c_i is less than π, we can shorten τ at c_i. Here the shorten technique can be seen in [4].

We now describe an algorithm for finding a δ-approximate path (DAP for short) joining a and b along the sequence of line segments (Algorithm 7 [24]).



(a) Input of the problem (b) An example of DAP (c) An example of DAP with $\delta = 1$

Figure 4.: δ-approximate path illustration

Given a sequence $\mathcal{F}$ of bundles of line segments, let ϵ_0 be a small positive number, we trim line segments of a bundle from the vertex a sub-segment ϵ long to obtain a sequence, say $\mathcal{F}^*$, of pairwise disjoint line segments from $\mathcal{F}$.

As shown in page 62 [24], Algorithm 1 runs in time $\frac{L_0 - L}{\epsilon} O(k)$, where L is the length of an optimal path and L_0 the length of the initial path.

Algorithm 1: The δ-approximate path $DAP(\mathcal{F}, a, b)$ - an approximate shortest path joining a and b along $\mathcal{F}$.

input : $\delta > 0$
Start point a
End point b
A sequence $\mathcal{F}$ of bundle of line segments such that at least one angle at some vertex of $\mathcal{F}$ is less than π
output: a δ-approximate path joining a and b along $\mathcal{F}$
Step 1: Initial a polyline P as sequence of points p_i in $d_i \in \mathcal{F}^*$;
Step 2: Find optimum position p_{inew} on its ce_i so that
$p_{inew} = \text{argmin}\{\|p_i - p_{i-1}\| + \|p_{i+1} - p_i\| : p_i \text{ in } d_i\}$;
Step 3: Replace p_i by p_{inew};
Step 4: Continue with vertices $p_{inew}, p_{i+1}, p_{i+2}$ then repeat step 3 until change path length is below δ.

3.2. The approximate shortest paths joining a pair of two points avoiding obstacles

Let $d_\epsilon \geq 0$ be the upper error bound for distances between any vertex $p_i \in d_i$ in the calculated path and its corresponding optimal vertex $p_i^* \in d_i$ in some iteration, which means that $\|p_i - p_i^*\| \leq d_\epsilon$, for all $d_i \in \mathcal{F}^*$. According to Theorem 3.3, p. 68 [24], Algorithm 1 obtains a δ-approximate path joining a and b with $\delta = 1+2(m+1)d_\epsilon/L$, where L denotes the length of the shortest path joining a and b, m is the number of line segments of $\mathcal{F}^*$. In addition, if for every sequence of bundle of line segments, angles at their vertices are greater than of equal to π, then τ is the shortest path joining a and b and avoids obstacles.

The demonstration of approximate path problem is shown in Figure 4. Workspace $\mathcal{W} = \mathcal{F}r_k \cup \mathcal{O}_k \subset \mathbb{R}^2$, where $\mathcal{F}r_k$ and $\mathcal{O}_k$ are disjoint free-known and obstacles-known (hatched gray polygons) space. The approximate shortest path problem computes a path P from start point $a(0,1) \in \mathcal{F}r_k$ to end point $b(4,3) \in \mathcal{F}r_k$ satisfying $P \in \mathcal{F}r_k$ and has minimum Euclidean length. In Figure 4b and 4c, the green area is visited and is a subset of $\mathcal{F}r_k$. The bundles of line segments ce_i are blue line segments.

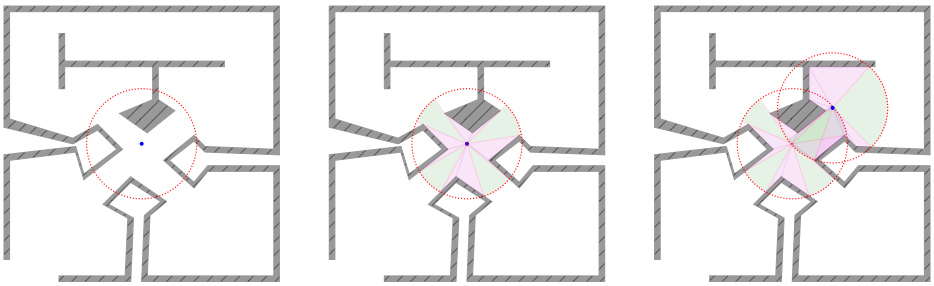
A δ-approximate path for p_{1new} of path $P = (p_0, p_1, p_2, p_3, p_4)$ where p_0, p_4 are start and end points, respectively in Figure 4b. Old path is dashed black path $P_{old} = (p_0, p_1, p_2, p_3, p_4)$ while new path is solid red path $P_{new} = (p_0, p_{1new}, p_2, p_3, p_4)$. The final result of DAP is the solid red path in Figure 4c.

4. Autonomous Robots

Let $\mathcal{W}$ denote working-space, $\mathcal{W} = \mathcal{F}r_k \cup \mathcal{O}_k \cup \mathcal{U} \subset \mathbb{R}^2$ where $\mathcal{F}r_k, \mathcal{O}_k, \mathcal{U}$ are disjoint sets denoting free-known, obstacles-known, and unknown space, respectively. $\mathcal{U} = \mathcal{F}r_u \cup \mathcal{O}_u$, unknown space consist of $\mathcal{F}r_u$ free-unknown and $\mathcal{O}_u$ obstacles-unknown. This paper assumes that $\mathcal{O}_k$ is static. This section depicts how an robot finds a path $\mathcal{P}$ which begins at start point $s \in \mathcal{F}r_k$ and ends at goal point $g \in \mathcal{F}r_u$

4.1. Robot's vision range and its neighbor and explored sights

Let C_t denote the position of the robot's center at $(x, y) \in \mathbb{R}^2$ at time t and $C_t \equiv C_{t+a}$ iff their coordinates are equality. Then $C_0 = s$ *start point*. In Figure 5a, obstacles are hatched polygons in gray, and the blue dot is a robot coordinate C_t. At particular C_t, the robot scans neighbor space around it in dashed red circle space $\mathcal{C}$ (in Figure 5b). The circle's center is C_t, and its radius is the robot's vision range r. $\mathcal{O}_k$ are obstacles-known in $\mathcal{C}$, $\mathcal{F}r_k$ is free-space known in $\mathcal{C}$, and $\mathcal{C} = \mathcal{F}r_k \cup \mathcal{O}_k$



(a) Work-space (b) Neighbor sight S_t^n at C_t (c) Explored sight S_{t+1}^e

Figure 5.: Work-space, robot's neighbor sights, and explored sights

At C_t, the robot scans around to get *neighbor sight* space (denoted by $\mathcal{S}_t^n$). The $\mathcal{S}_t^n \in \mathcal{F}r_k$ consists of closed sights and open sights. The closed sights are visibility triangles from the point C_t and limited by the boundaries of visibility obstacles $\mathcal{O}_k$, whereas the open sights are visibility arcs from point C_t and limited by circle $\mathcal{C}$. In Figure 5b, triangles in pink are closed sights, and green arcs are open sights. Let $\mathcal{S}_t^e$ denote the *explored sight* and is the union of *neighbor sights*. So $\mathcal{S}_t^e = \cup(\mathcal{S}_t^n) \subset \mathcal{F}r_k$. Figure 5c demonstrates the *explored sight*, that is the union of all pink triangles and green arcs. When the robot moves from at initiative coordinate C_t to the next coordinate C_{t+1}, the *explored sight* will be expanded.

4.2. The direction of the robot

Once the robot gets $\mathcal{S}_t^n$ at $C_t = (x, y)$, it finds directions leading to the goal g. This part describes how the robot determines such directions.

At particular coordinate, directions from the robot's center C_t to somewhere in closed sight space are collision directions, while directions from C_t to somewhere in open sights space are collision-free directions.

As shown in Figure 6, the sectors of C_t corresponding to the green triangles illustrate open sights that robot can move in the next steps whereas the pink triangles demonstrate closed sights that give collision warnings. Each open sight has the so-called *open point* which is the midpoint of each arc with respect to the sector determining one open sight. Because moving to the directions from the center a to a point in closed sights, the robot will hit some obstacle, at the next step, it must find one open point to move to.

Let open point p represent the direction of an open sight at $C_t = (x, y)$. Figure 6 illustrates *open points* of the robot at different locations where orient to goal g (red star point). At $C_0 = (5, 0)$, there are four open points (in blue and red) corresponding to three directions of the robot at that location. The red one is considered a higher

priority than the blue ones. We discuss the ranking of open points in Subsection 4.3. If the robot is at $C_2 = (50, 50)$, there are four ways to go since the robot detects four open points. Let O_t^n denote a set of open points p at C_t. When the robot moves from C_0 to C_t, there are sets of open points O_t^n for each C_t. Let O_t^g be a union of O_t^n, so $O_t^g = \cup(O_t^n)$ is a global open point set that robot needs to consider.

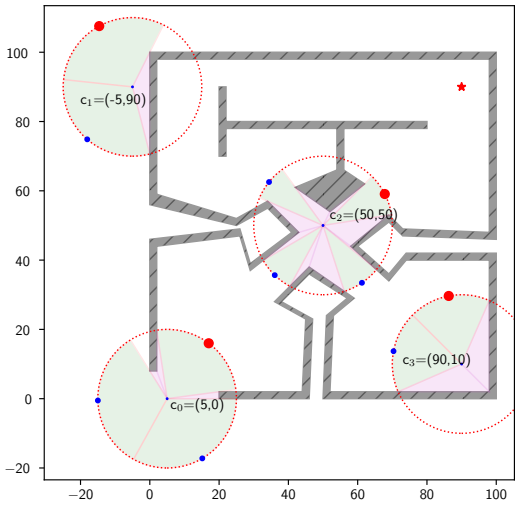


Figure 6.: Open points at different locations.

4.3. The control of the robot

Once the robot has many open points $p \in O_t^g$ as candidates, it needs to choose which open point it should follow to find the goal. This part demonstrates how the robot scores open points.

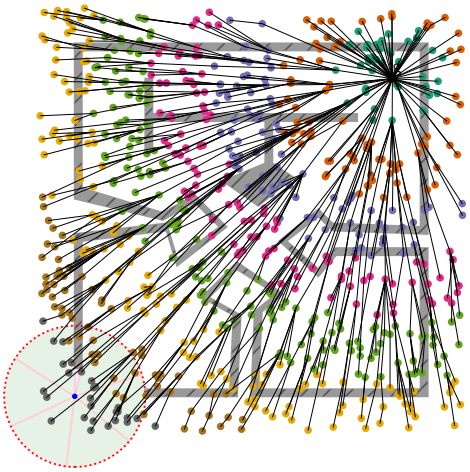


Figure 7.: A ranking tree of 500 nodes.

When the robot is at new coordinate C_t, it scans around and gets *open points* $p \in O_t^n$. If $p \in \mathcal{S}_t^e$, p is inactive point, then the robot ignores them. If $p \notin \mathcal{S}_t^e$, p is *active open points*. The robot estimates the ranking of those *active open points*. The higher the ranking of an open point is, the more priority it is. To score an open point p, the robot uses two factors: distance and angle. Hence, our robot applies the formula

$$ranking(p, C_t) = \begin{cases} \frac{\alpha}{d(p,g)} + \frac{\beta}{a(p,C_t,g)} & : d(p,g) \neq 0 \text{ and } a(p,C_t,g) \neq 0 \\ +\infty & : d(p,g) = 0 \text{ or } a(p,C_t,g) = 0 \end{cases} \quad (1)$$

to score, where $d(p, g)$ is $\mathbb{R}^2$ Euclidean distance between point p and g, and $a(p, C_t, g)$ is angle $\angle pC_tg$. All open points, which are scored at a specific coordinate C_t, are stored in a set called *neighbor rank* $\mathcal{R}_t^n$. So $\mathcal{R}_t^n = ranking(p, C_t)|p \in O_t^n$. And the union of all $\mathcal{R}_t^n$ is *explored rank* $\mathcal{R}_t^e = \cap\mathcal{R}_t^n$. Based on either *neighbor rank* $\mathcal{R}_t^n$ or explored rank $\mathcal{R}_t^e$, our system will develop picking strategies to find the better robot's path.

The next point where the robot will move to is a point $p \in (O_t^g)$ satisfy its ranking $= \text{argmax}\{x \in \mathcal{R}_t^e\}$. When the robot already visited an open point $p \in O_t^g$, it updates $O_t^g = O_t^g \setminus p$ and $\mathcal{R}_t^e = \mathcal{R}_t^e \setminus x$

4.4. Alternative open points and ranking formula

The robot also uses a ranking tree, namely $\mathcal{T}(V, g, n)$ to score open points, where V are $\mathcal{T}$'s vertices, g is the given goal, n is the number of $\mathcal{T}$'s nodes. $\mathcal{T}(V, g, n)$ is an RRT*-liked (Rapidly-exploring random trees star), where its root is at g, and each ranking tree's node $v_i \in V$ contains an estimated cost-to-goal c_g^i. If v_i is a parent of v_j , the cost-to-goal for v_j is calculated by $c_g^j = c_g^i + d(i, j)$, where $d(i, j)$ is $\mathbb{R}^2$ Euclidean distance between nodes i and j, and $c_g^i = 0$, iff $v_i \equiv g$. At a specific coordinate C_t, nodes are inside open sights and will be considered as open points. It means an open sight contains many open points. To reduce the number of open points, it selects only one node, which is the most far away from C_t as an *open point* p for its open sight. We have $p = \text{argmax}\{d(x, C_t)|x \in (V \cup \mathcal{C})\}$ where $d(x, C_t)$ is a distance between x and C_t. Our robot applies a formula to score an open point p at the tree node:

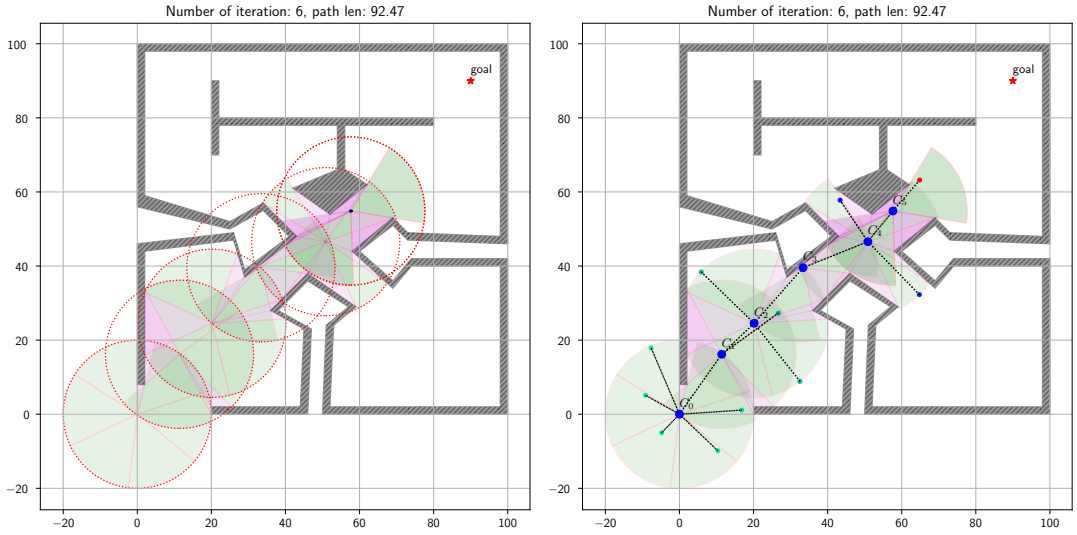
$$ranking(p) = \begin{cases} \frac{1}{c_g^p} & : c_g^p \neq 0 \\ +\infty & : c_g^p = 0 \end{cases} \quad (2)$$

Figure 7 depicts a ranking tree with 500 nodes for goal $g = (90.0, 90.0)$. At $C_0 = (0.0, 0.0)$, where the vision range is a shaded circle, the robot has six open sights. Each open sight will pick a maximum-distance node (in red) from its center to represent its open point.

4.5. The visibility graph and optimal paths of the robot w.r.t. length, computational time, and number of turns of of traveled paths

In reality, when a robot works towards a goal, it must take into account several factors that can affect its battery life and overall performance. These factors include

the lengths, computational time, and fewest turns of of traveled paths. This section discusses how our robot minimizes these factors to optimize its performance. In experimental results, we will use these factors to evaluate and compare our results.



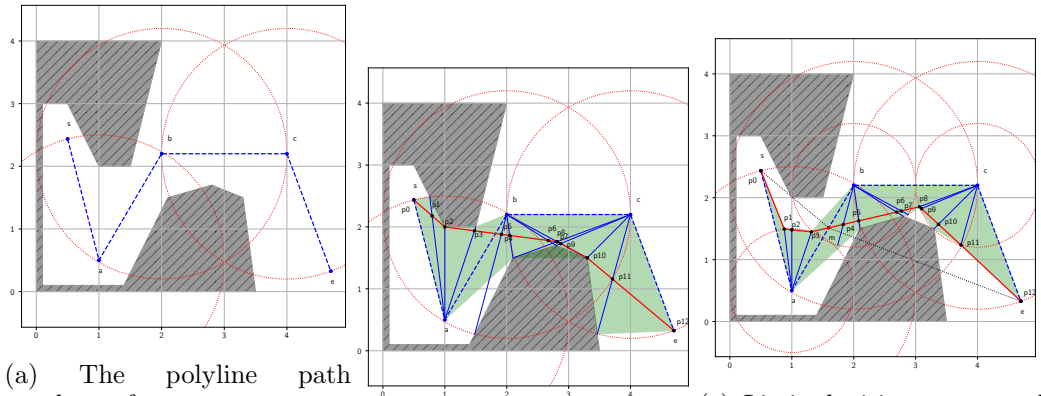
(a) The robot's explored sight $\mathcal{S}_t^e$ (b) The robot's visibility graph $\mathcal{G}_t^e$

Figure 8.: Robot moves from C_0 to C_5.

At $C_t = (x, y)$, the robot gets active open points $p \in O_t^n$. There exists a neighbor graph $\mathcal{G}_t^n$ where its vertices $V_t^n = \{C_t, p\}|p \in O_t^n\}$, and its edges $E_t^n = \{(C_t, p)|p \in O_t^n\}$. Because $\mathcal{G}_t^n \in \mathcal{F}r_k$, therefore $\mathcal{G}_t^n$ is a collision-free graph. Let $\mathcal{G}_t^e$ is union set of $\mathcal{G}_t^n$. After which, the robot moves from C_t to the next point $C_{t+1} = p \in O_t^g$. We have $C_{t+1} \in \mathcal{G}_{t+1}^n$ and $C_{t+1} \in \mathcal{G}_t^e$. Therefore, the set $\mathcal{G}_t^e = \mathcal{G}_{t-1}^e \cap \mathcal{G}_t^n$ is a connected graph and $\mathcal{G}_t^e \in \mathcal{F}r_k$. Figure 8a presents an explored sight since the robot moves from C_0 to C_5 and Figure 8b shows the visibility graph (in black dashed line segments) of that sight. There are six local graphs where their roots are at $C_0, C_1, \ldots, C_5$. At C_0 there are six local active open points scored by ranking tree $\mathcal{T}$ as six directions to go. At C_1 there are three open points by ranking tree $\mathcal{T}$ but one is inactive because that open point is in the explored sight.

Once the robot picks the next point based on the ranking, the robot finds a path from the robot's current to the next point. If the next point $C_{t+1} \in \mathcal{S}_t^n$, the robot just goes straight. If the next point $C_{t+1} \in \mathcal{F}r_n \setminus \mathcal{S}_t^n$, we apply Algorithm 1 to compute an approximate shortest path effectively.

In term of complicated polyline path $sp = (s, a, b, c, e)$ in Figure 9a, the bundle of line segments from b joints with the bundle of line segments from d. The sequence of bundles of line segments makes basic collision-free area invalidated as shown in Ref 9b. To satisfy the assumption that line segments of bundles of disjoint vertices do not intersect each other, we disjoint line segments each other by shortening them. By Algorithm 1, the final approximate shortest path $P = (p_0, \ldots, p_{11})$ is shorter than the path $sp = (s, a, b, c, d)$ (see in Figure 9c). So (BAR) problem has been solved: the robot at s finds the shorter path from s to e to escape the blind alley region formed by shaded domain (obstacle).



(a) The polyline path s, a, b, c, e for an autonomous robot to escape from the blind alley region.

(b) The invalidated collision-free area.

(c) Limited vision range and basicthe corresponding sequence of bundles of line segments

Figure 9.: The approximate shortest path joining $a = p_0$ and $b = p_{12}$ along the sequence of bundles of line segments.

4.6. The operation of autonomous robots

In this subsection, we combine all the above solutions to build an algorithm for autonomous robot operations including solving (BAR) problem to reach the goal.

First, the start s and goal g are set for the autonomous robot. Then the robot starts at $C_0 = s$ and scans around to get $\mathcal{S}_o^n$ and active *open points* set O_0^n. After that, the robot estimates the ranking $\mathcal{R}_0^n$ for all active open points at C_0. Next, it expands *explored open points* $O_t^e = O_{t-1}^e \cup O_t^n$, explored ranking $\mathcal{R}_t^e = \mathcal{R}_{t-1}^e \cup \mathcal{R}_t^n$, explored sights $\mathcal{S}_t^e = \mathcal{S}_{t-1}^e \cup \mathcal{S}_t^n$, and neighbor graph $\mathcal{G}_t^e = \mathcal{G}_{t-1}^e \cup \mathcal{G}_t^n$, where $O_{-1}^e, \mathcal{R}_{-1}^e, \mathcal{S}_{-1}^e$ are empty set. Afterward, the robot picks the next point n where $n \in O_t^g$ and n satisfies $\text{argmax}\{x \in \mathcal{R}_t^e\}$. Before finding the approximately shortest path, the robot finds the polyline path in the graph $\mathcal{G}_t^e$. It then determines the sequence $\mathcal{F}$ between C_t and n from $\mathcal{G}_t^e$ and then the robot moves to the next point n on the approximately shortest path between C_t and n along $\mathcal{F}$ and repeats the above process until the robot meets the goal.

Figure 10 demonstrates how an autonomous robot moves to a goal $g = (40, 90)$ from start $s = (0, 0)$. At $C_0 = (0, 0)$, the robot gets neighbor sight and determines six open points as shown in Figure 10a. The red point is the next point n because its ranking score is greater than the others. The remaining points will store in active points set O_t^g. After moving to the next point, the robot scans neighbor sight at the next point n and gets two more active open points (in red and blue) as shown in Figure 10b. The red point is the next point now because its ranking is highest in O_t^g. The robot keeps exploring the place as shown in Figure 10c, and follows Algorithm 2 until the robot reaches the goal as shown in Figure 10d.

Our DAP length depends on the visibility graph $\mathcal{G}_t^e$, sometimes it is not good because our $\mathcal{G}_t^e$ represents points where our robot visited. Therefore we improve our DAP length by adding more connections inside $\mathcal{G}_t^e$. First, when the robot goes to new coordinate C_t, it will consider $C_i \in C_t^e | d(C_i, C_t) < 2r$ where r is vision range. If there exists an intersection S_j where $S_j \in C_t$'s sight and $S_j \in C_i$'s sight, we find a center point C_j of S_t and update visibility graph $V_g = \{V_g \cup C_j\}$, $E_g = \{E_g \cup (C_t, C_j) \cup (C_i, C_j) | (C_t, C_i, C_j) \in V_g^3\}$. Because S_j is both in C_t's and C_i's sight, therefore the new connection is free-collision edge.

Algorithm 2: Autonomous Robot Operation Algorithm

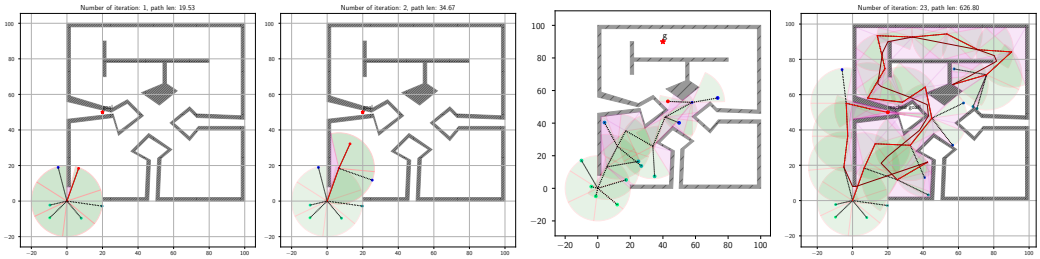
input : Start s;
Goal g;
Polygonal obstacles $\mathcal{O}$;
output: A path $\mathcal{P}$ from s to g
robot coordinate $C_t \leftarrow s$;
explored sight $\mathcal{S}^e \leftarrow \emptyset$;
explored open points $\mathcal{O}^e \leftarrow \emptyset$;
explored ranks $\mathcal{R}^e \leftarrow \emptyset$;
path $\mathcal{P} \leftarrow s$;
while *True* **do**
  if $C_t \equiv$ *goal* g **then**
    $\mathcal{P} = \mathcal{P} \cup g$;
    return *True* ;
  else
    get neighbor open points O_t^n;
    get neighbor graph $\mathcal{G}_t^n$;
    neighbor rank $\mathcal{R}_t^n \leftarrow \emptyset$;
    for *each open point* p *in* O_t^n **do**
      if p *is active open point* **then**
        $\mathcal{R}_t^n = \mathcal{R}_t^n \cup ranking(p)$;
    end
    $O_t^g = O_{t-1}^e \cup O_t^n$;
    $\mathcal{G}_t^e = \mathcal{G}_{t-1}^e \cup \mathcal{G}_t^n$;
    $\mathcal{R}_t^e = \mathcal{R}_{t-1}^e \cup \mathcal{R}_t^n$;
    next point $n \in O_t^g$ satisfies $ranking(n) = \text{argmax}\{(\mathcal{R}_t^e)\}$;
    if $n \equiv \emptyset$ **then**
      return *False* ;
    else
      (now a blind alley region occurs, we then deal with a (BAR) problem);
      Extract from $\mathcal{G}_t^e$ the sequence $\mathcal{F}$ between C_t and n;
      Call $DAP(\mathcal{F}, C_t, n)$ to find approximately shortest path DAP along $\mathcal{F}$;
      Make a move form C_t the next point n along DAP;
      Remove n from O_t^g;
      $O_t^g = O_t^g \setminus n$;
      $\mathcal{R}_t^e = \mathcal{R}_t^e \setminus ranking(n)$;
      $\mathcal{P} = \mathcal{P} \cup DAP$;
      $C_t \leftarrow n$;
    end
  end
end

Some advantages of Algorithm 2 are that

a) It reduces search space because it searches locally,
b) At each blind alley region, it provides an efficient path DAP between 2 points

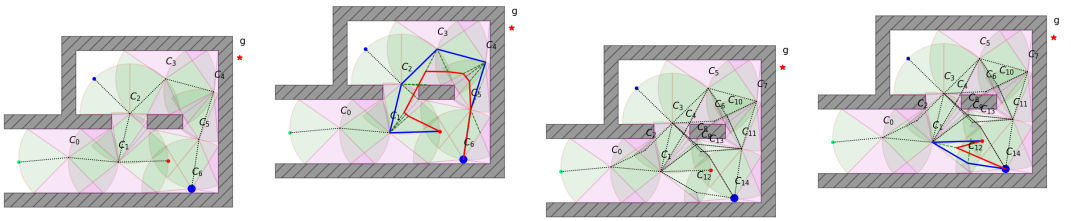
Term	Description
s	start point
g	goal point
$\mathcal{P}$	a path from s to g which contents set of points
C_t	current robot's coordinate
$\mathcal{S}_t^n$	neighbor sights at point C_t
$\mathcal{S}_t^e$	explored sight at time t
O_t^n	neighbor open points at C_t
O_t^g	open points
$\mathcal{R}_t^n$	neighbor rank at point C_t
$\mathcal{R}_t^e$	explored ranks at time t
$\mathcal{G}_t^n$	neighbor graph at point C_t
$\mathcal{G}_t^e$	visibility graph at time t
n	next point
SP	shortest polyline path from robot's coordinate C_t to n
DAP	δ-approximate path from current robot's coordinate

Table 1.: Symbols used in Algorithm 2



(a) Step 1 (b) Step 2 (c) Step 7 (d) Result

Figure 10.: Autonomous robot operation



(a) $\mathcal{G}_t^e$ (original) (b) ASP on $\mathcal{G}_t^e$ (original) (c) $\mathcal{G}_t^e$ (improve) (d) ASP on $\mathcal{G}_t^e$ (improve)

Figure 11.: Improve visibility graph by adding possible connection among the visibility graph's vertices

in a sequence of bundles of line segments that is a simple explored space. As a result, the (BAR) problem is solved.

c) Due to the construction of the sequences of bundles of line segments, such paths DAP are not always "close" obstacles and the number of turns of such paths is

smaller ones determined by other shortest path algorithms (e.g., A*, RRT*).

These can be seen later in Section 6.2.

A disadvantage of Algorithm 2 is that it might not be convergent. However, in the next section, we show the convergence of the algorithm under some assumptions.

5. A Convergent Algorithm and its Correctness

We now consider a special case of Algorithm 2. Under some assumptions of the obstacles and the ranking (1), the corresponding algorithm is convergent with respect to $\|.\|$ and then allows a robot to reach the goal.

Assume that

$$\text{when } c \in B(g,r), \qquad \text{there is a path avoiding obstacles and joining } c \text{ and } g. \quad (3)$$

The path given in (3) can be made by someone of the Bug Family Algorithm. In this section, we assume the obstacle $\mathcal{O}_k$ consists of the distinct compact sets OB_i, $i = 1, \cdots, k$ with

$$\text{diam}OB_i \quad < \quad r\sqrt{3} \tag{4}$$

for all $i = 1, \cdots, k$. Set

$$\mathcal{M}(c) \quad := \quad \text{bd}\bar{B}(c,r) \cap B(g, \|c-g\|) \tag{5}$$

(see Figure 12 with $\mathcal{O}_k = OB_1 \cup OB_2$).

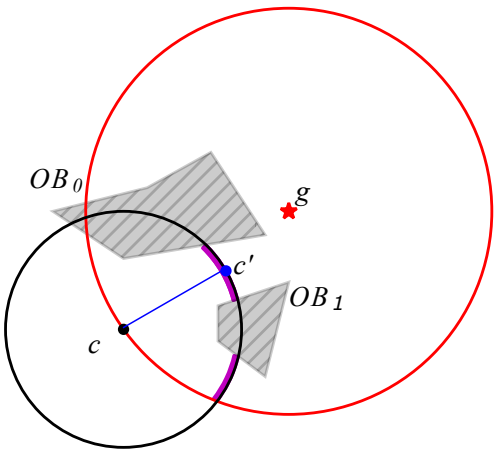


Figure 12.: The set $\mathcal{M}(c)$ consists of two brown arcs.

Given a robot at the location c, let us define the new location c' for the robot as follows

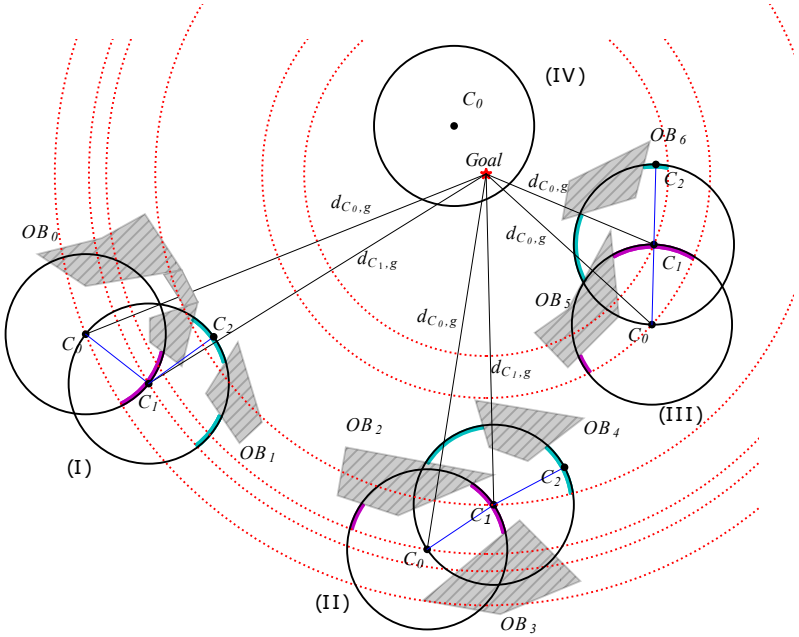


Figure 13.: Cases (I)-(III): Definition of the point $c' \in \mathcal{M}(c)$ ($c'' \in \mathcal{M}(c')$, resp.) when the robot at c (c', resp.) avoids the obstacle (colored in blue). Case (IV): The robot reaches the goal g.

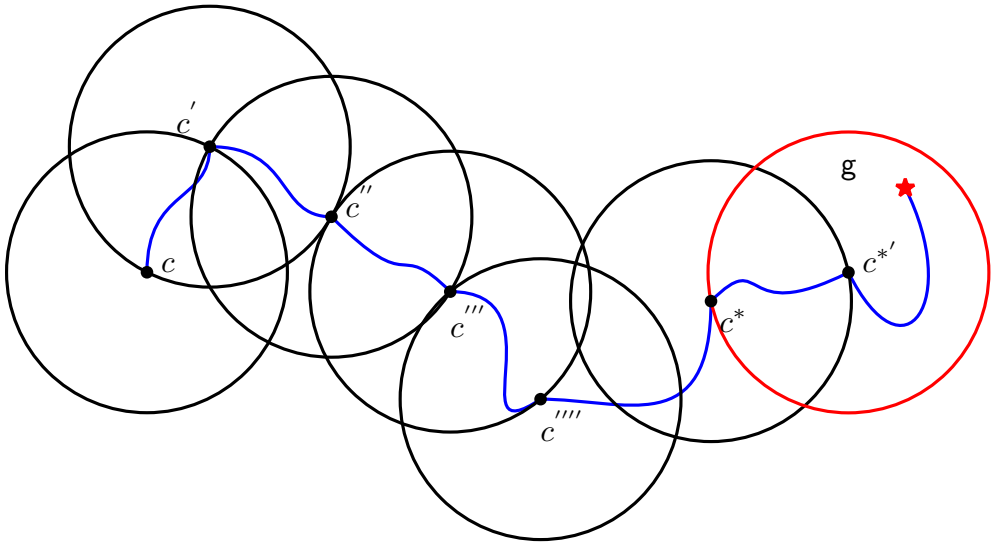


Figure 14.: The robot goes from c through c^* and then through $c^{*\prime}$ to the goal g.

$$c' \quad \in \quad \text{argmin}\{\|m - g\| : m \in \mathcal{M}(c)\}. \tag{6}$$

By (4), $\mathcal{M}(c) \neq \emptyset$ is obtained. Since P is compact, such c' exists and there are at most two such ones. Figure 13 illustrates cases (I)-(III) to define $\mathcal{M}(c)$, the new location c', and $\mathcal{M}(c')$ (they are brown arcs. In case (IV), the robot reaches the goal g. The Algorithm 3 below is under the assumptions (3) and (4). For (3), paths joining two points avoiding obstacles are line segments. In Figure 15, with an obstacle colored in black, the path done via the bug algorithm is longer than the one created by Algorithm 3 (AR).

Algorithm 3: Autonomous Robot (AR)

input : Start s
Goal g
Polygonal obstacles
output: a path P from s to g
Current Coordinate $c \leftarrow s$;
$i = 0, c_0 := s$;
Path $P \leftarrow s$;
(*) Consider c_i ;
if $\|c_i - g\| > r$ **then**
  Take $c_{i+1} \in \mathcal{M}(c_i), i = i++$;
  goto (*);
else
  Take $R(c_i, g)$ (determined by (3)).
end
The union of $R(c_0, c_1), \ldots, R(c_{i-1}, c_i), R(c_i, g)$ joins c and g.

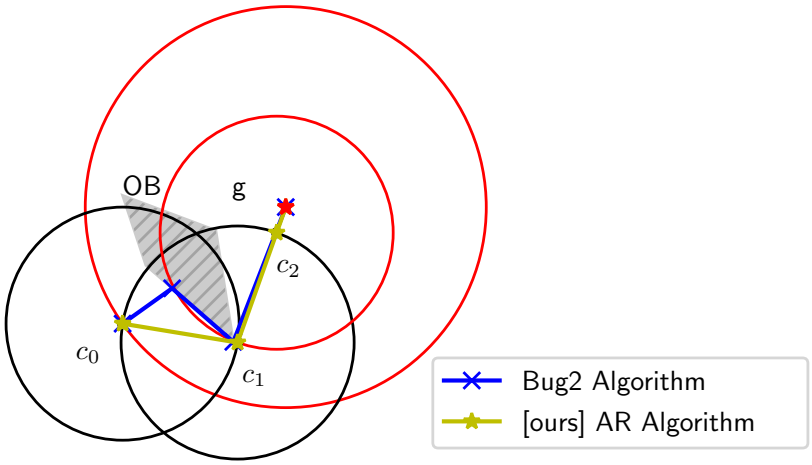


Figure 15.: The robot goes from $c = c_1$ through c_2 and then through c_3 to g. The path done via bug algorithm colored in blue is longer than the one coloured in yellows via by AR

We now consider the correctness of Algorithm 3.

Proposition 5.1. *Under the assumptions (3) and (4), Algorithm 3 allows the robot*

reaches the goal after a finite number of steps.

Proof. If the robot at c and $g \in \bar{B}(c,r)$ then it can reach the goal g avoiding any obstacle satisfying (4) in a finite number of steps. Consider the robot at c and $g \notin \bar{B}(c,r)$. By (4), we have $\mathcal{M}(c) \neq \emptyset$. By (5), $\|c'-g\| < \|c-g\|$ and therefore $\|c''-g\| < \|c'-g\|$ if $g \notin \bar{B}(c',r)$. It follows that at ith step, the algorithm gives a decreasing sequence of positive numbers $\{\|c'^i - g\|\}$ (see Figure 14). If the algorithm has an infinite number of steps. Then, $\{\|c'^i - g\|\}$ converges to some positive number, say γ. $g \notin \bar{B}(c'^i, r)$ implies $\gamma \geq r$. Then, $\{c'^i\}$ converges to c^*. Because of the convergence of $\{\|c'^i - g\|\}$, we obtain $\gamma = r$. Thus, there exists $c^* \in \bar{B}(g,r)$ such that $\{c'^i\}$ converges to $\{c*\}$. By (5), $\|c^{*\prime} - g\| < \|c^* - g\| = r$ with some $c^{*\prime} \in \mathcal{M}(c^*)$. Clearly, the robot at c^* (at $c^{*\prime}$, respectively) can reach $c^{*\prime}$ (to the goal g, respectively) avoiding any obstacle satisfying (4). It follows that we can take a finite number of steps, the robot goes from the start location through c^* and then $c^{*\prime}$ to g. □

6. Experimental Results

6.1. Experimental setup

We design two experiments to evaluate the performance of the proposed algorithm in different scenarios. The first experiment including (BAR) problems, aims to see how good our feasible paths (i.e., that reach the given goal avoiding obstacles) are, compared to the A* and RRT* paths in the explored area w.r.t. the length of path, the computational time for finding the path, and the number of turns. The second experiment including (BAR) problems, aims to compare the feasible paths between our proposal and RRTX. We check how many feasible paths and the length of those paths. We use two environments, namely blind alley regions 1 and 2, included in (BAR) problems, for testing as shown in Figure 2. The code written in Python could be found at: https://github.com/ThanhBinhTran/autonomousRobot.

In the first experiment, while exploring an unknown area, the robot moves from $C_t \in \mathcal{F}r_k$ to $n \in \mathcal{F}r_k$. If the next point $n \in \mathcal{S}_t^n$ (the neighbor sight of C_t), the robot just goes straight from C_t to n and does not compare ours with the A* or RRT*. If $n \in \mathcal{F}r_k \setminus \mathcal{S}_t^n$, the robot finds an approximated path in visited sight $\mathcal{S}_t^e$. In the latter cases, the system records results for comparison. In particular, our ASP search on visibility graph $\mathcal{G}_t^e$ to find the shortest path SP from $C_t \in \mathcal{G}_t^e$ to $n \in \mathcal{G}_t^e$ then we determine bunches of line segments alongside with SP before applying Algorithm 1 to get ASP. Regarding A*, we provide grid G_{A*} with cell size (= robot size) and then use A* to find the shortest path from $C_t \in G_{A*}$ to $n \in G_{A*}$. In terms of RRT*, we are given 2000 nodes of RRT* with RRT*'s root is C_0 and all open points $p \in O_t^g$ satisfying $\forall p \in O_t^g, p$ is RRT*'s nodes. And find a path from C_t to n in RRT*. We carry out this experiment in both maps with fixed start point $s = (0,0)$ and pick different goal points $g = (i * step_size + 20, j * step_size + 20) | i, j \in \{1, .., 9\}$ where $step_size = 10$ for blind alley region 1, and $= 20$ for blind alley region 2. In cases, goal $g \in \mathcal{O}$ (Obstacles), we skip these invalid goal.

In the second experiment, we experience a comparison between our algorithm and RRTX in terms of total traveled path length under the same circumstance (e.i., map, start point, goal point, number of tree nodes) on both maps. Similarly, we fixed the start point $s = (0,0)$ and pick different goal points $g = (i*step_size+20, j*step_size+20) | i, j \in \{1, .., 9\}$ where $step_size = 10$ for blind alley region 1, and $= 20$ for blind alley region 2. In cases, goal $g \in \mathcal{O}$ (obstacles), we skip these invalid goal. For each pair

of (s, g), we change vision range r from 10 to 30 with a 5-unit step size range (equality 1 to 3.0 LiDAR meter in reality, respectively). We design two picking strategies: pick global first and pick neighbor first. In the picking global first strategy, all new neighbor open points are merged into a global open point set then the next point n will be picked in the global set, whereas the picking neighbor first strategy selects n in a new neighbor set first. If the new neighbor set is empty, the system selects n in the global set.

6.2. Experimental results

6.2.1. Experiment 1

Figure 16, 17 show some images captured in the comparison for Experiment 6.2.1. In Figure 16, the robot moves from start point $s = (0, 0)$ to goal (red star point) $g(70, 90)$. There are two cases associated with two rows where the robot moves from C_t to the next point (in red) $n \notin \mathcal{S}_t^n$ in blind alley region 1. In each row (from left to right), the red paths are a generated trajectory of [our] improved ASP, A*, and RRT*, respectively. Similarly, in Figure 17, there are three cases when the robot moves from $s = (0, 0)$ to $g = (160, 80)$ in blind alley region 2.

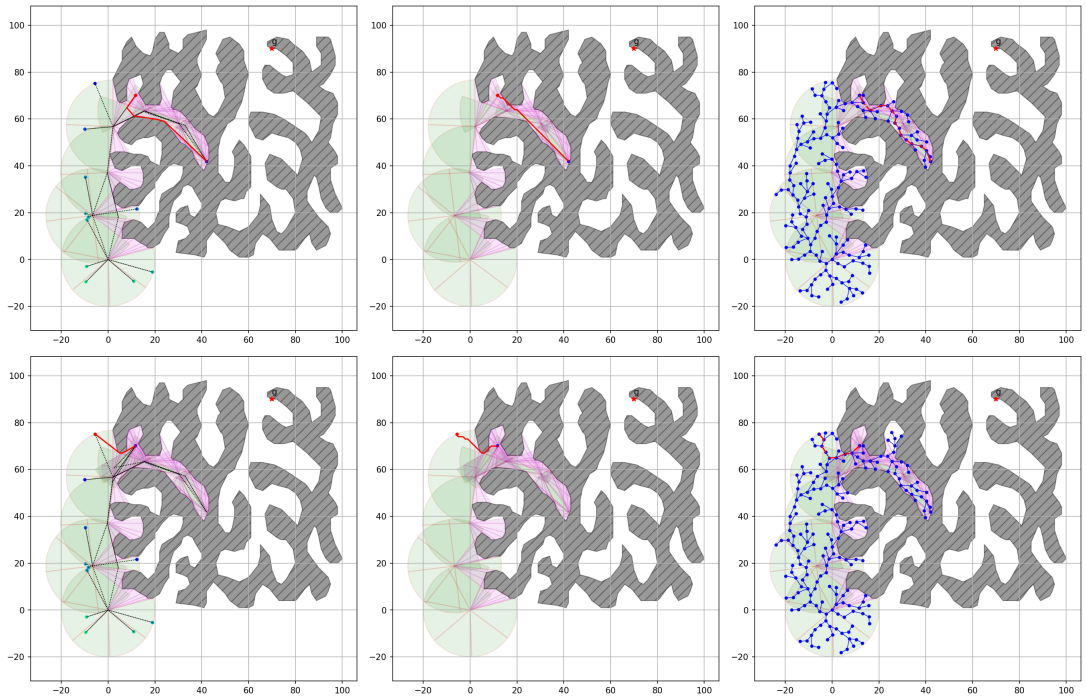


Figure 16.: Composite image of traveled path among [our] improved ASP, A*, and RRT* (from left to right); start s =(0, 0), goal g =(70, 90). The path obtained by our proposal is always shorter than the one obtained by RRT*.

For blind alley region 1, Figure 16 compares traveled paths length among our proposal, A*, and RRT*. In each valid goal, it calculates the mean of length of traveled paths of all cases where the robot moves from $C_t \in \mathcal{F}r_k$ to $n \in \mathcal{F}r_k \setminus \mathcal{S}_t^n$. The paths obtained by our proposal are always shorter than the ones obtained by RRT*.

Similarly, Figure 19 shows the mean of computational time for finding the feasible

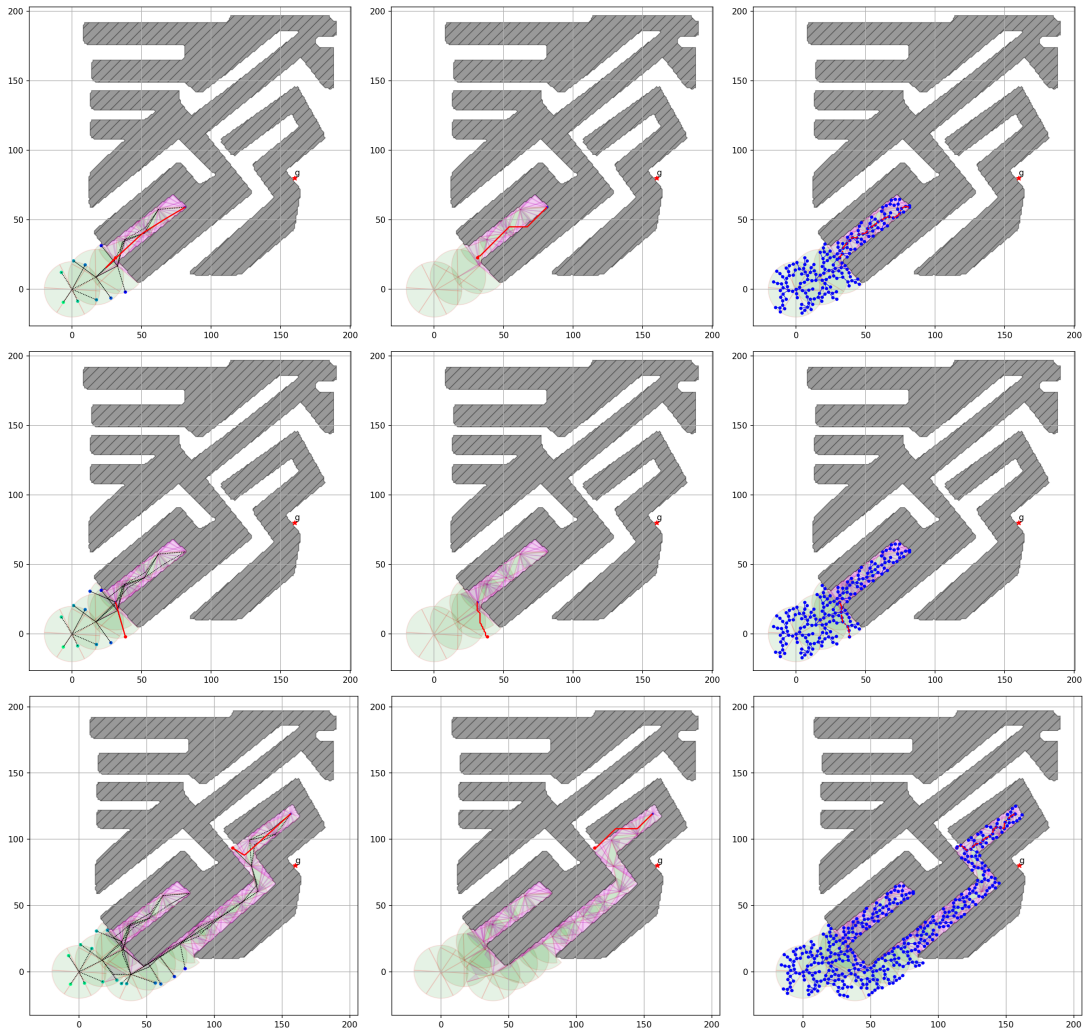


Figure 17.: Compositing image of traveled path among [our] improved ASP, A*, and RRT* (from left to right); start s =(0, 0), goal g =(160, 80).

paths and Figure 23 presents the number of turns of the traveled paths. In general, the number of turns obtained by our proposal is smaller than the ones obtained by A* and RRT*. In detail, the mean of our path cost is 38.638975 (m). That number of A* and RRT* are 33.084405 (m) and 45.830515(m), respectively. Regarding computational time, 0.024384(s), 0.010619 (s), and 0.001294 (s) are time results of our proposal, A*, RRT* on average, respectively (Figure 19). In terms of the number of turns in traveled paths, our proposal provides smoother paths with 3.590554 on average compared to 7.303526 of A* and 12.340282 of RRT* (in Figure 23).

For blind alley region 2, Figure 21 compares traveled paths length among our proposal, A*, and RRT* and presents the number of turns of the traveled paths. The one obtained by our proposal is the smallest. In detail, the mean of our path cost is 65.735244 (m). That number of A* and RRT* are 58.964694 (m) and 78.073804(m), respectively (Figure 21). Regarding computational time, 0.089816 (s), 0.046497 (s), and 0.001432 (s) are time results of our proposal, A*, RRT* on average, respectively (Figure 23). In term of number of turns in traveled paths, our proposal provides smoother

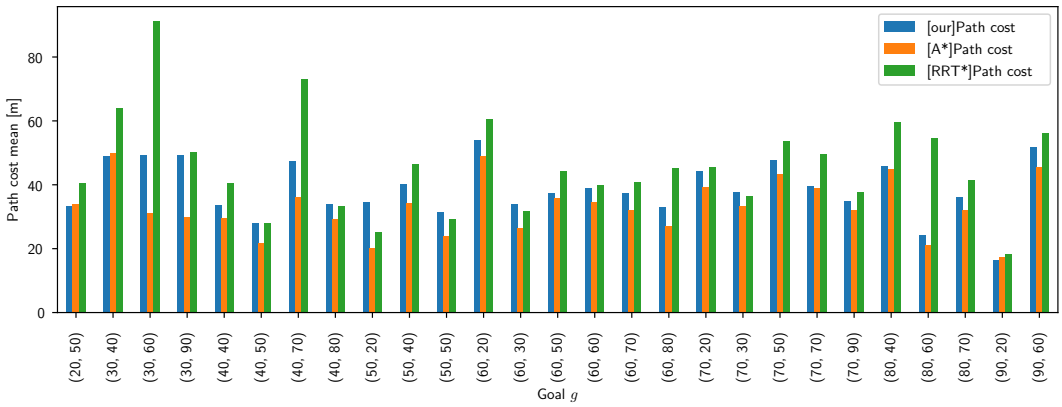


Figure 18.: Comparison between the lengths of traveled paths among our proposal, A*, RRT* on blind alley region 1. Our one is always shorter than the ones obtained by RRT*.

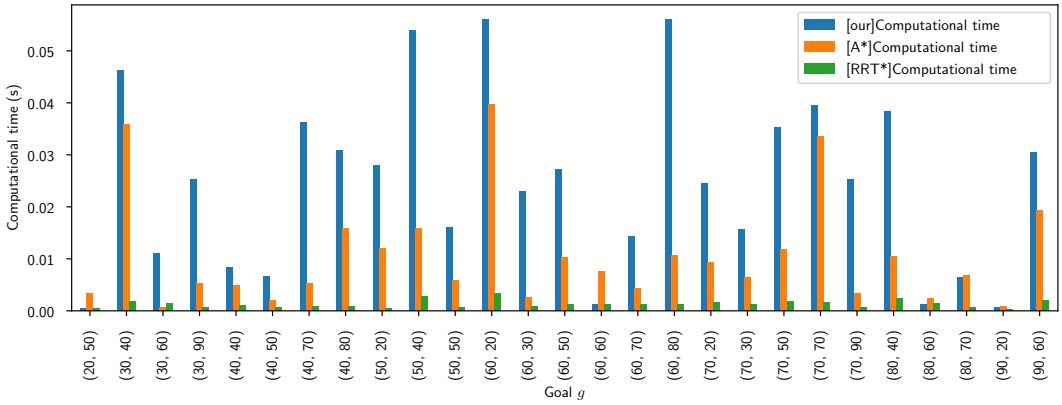


Figure 19.: Comparison between computational time of providing feasible paths among our proposal, A*, RRT* on blind alley region 1. In term of number of turns in traveled paths, our proposal provides smoother paths.

paths with 12.693362 on average compared to 14.380360 of A* and 21.909323 of RRT* (Figure 23).

6.2.2. Experiment 2

Figure 24, 25 show results of the whole generated trajectory of our proposal and RRTx. In Figure 24, there are five rows and three columns of image array, which display results for blind alley region 1. The first column demonstrates our improved travel path with the global set first picking, the second column presents our improved travel path with the neighbor set first picking, and the last column shows the traveled path of RRTX.

For the blind alley regions, the results of the number of feasible paths and the total path length are shown in Figure 26, 27 for all valid goals. In each valid goal, it calculates the mean of goal-reached paths of all ranges where the robot moves from start s to goal g. For all valid goals where our proposal and RRTX both provide

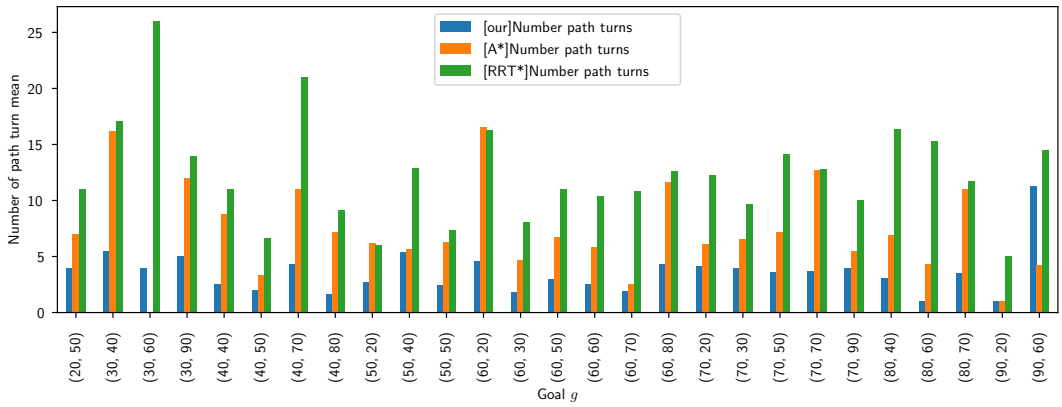


Figure 20.: Comparison between number of turns of traveled paths among our proposal, A*, RRT* on blind alley region 1. Our number is always smaller than the one of RRT*.

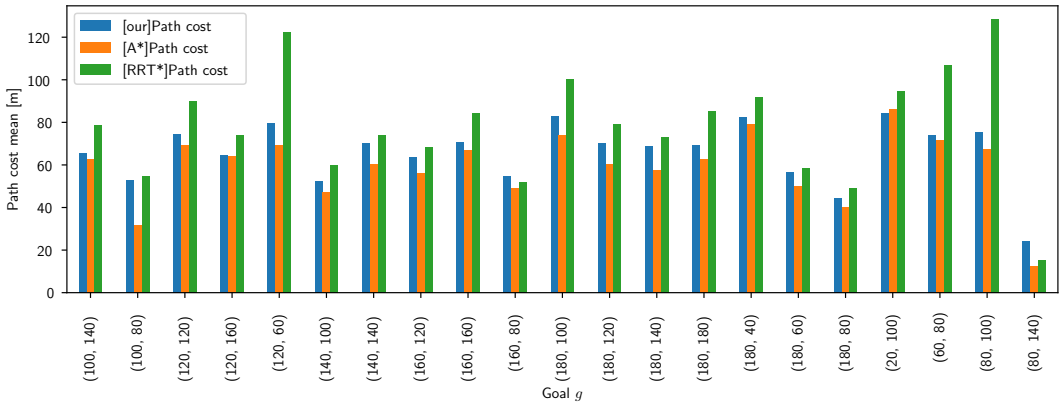


Figure 21.: Comparison between the length of traveled paths among our proposal, A*, RRT* on blind alley region 2. Our one is shorter than RRT*'s one.

feasible paths for all robot vision ranges, it calculates the mean of the total length of the feasible paths. Figure 27 shows the result of the feasible paths.

Similarly, for blind alley region 2, the results of the number of feasible paths and the total path length are shown in Figure 28, 29 for all valid goals. In each valid goal, it calculates the mean of goal-reached paths of all ranges where the robot moves from start s to goal g. The mean of our feasible paths is 3.903226 times for picking global first and 3.870968 times for picking neighbor first. That number of RRTX is 3.870968 times (Figure 28) lightly bigger than our ones.

Consider the special case presented in Sect. 5: obstacles satisfy the assumptions (3) and (4). In Figure 32-33, robot's vision is $r = 100$, start point $s = (140, 70)$ and goal point $g = (350, 350)$. Algorithm 3 gives the red dot polyline in Figure 33. The vertices of the red dot path are locations that the robot goes through. The red dot line segments are only pseudo path. The path that the robot goes through two such vertices is given by (33).

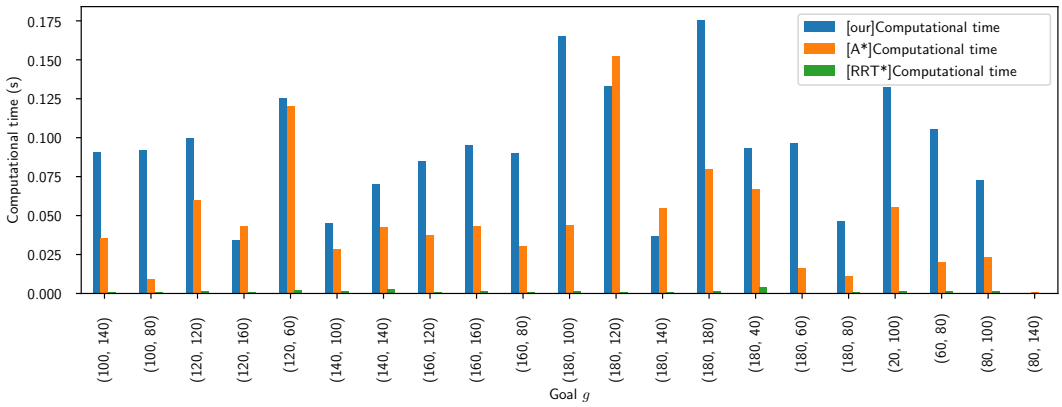


Figure 22.: Comparison between computational time of providing feasible paths among our proposal, A*, RRT* on blind alley region 2.

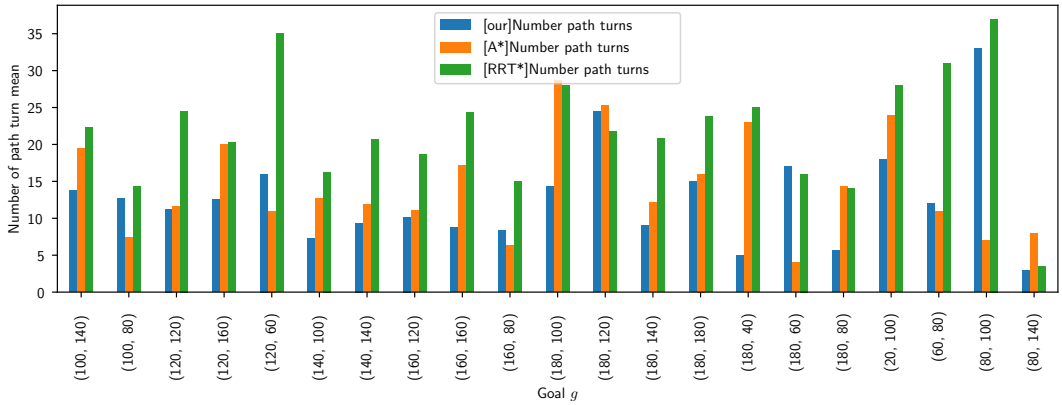


Figure 23.: Comparison between number of turns of traveled paths among our proposal, A*, RRT* on blind alley region 2. Our one is smaller than the one obtained by RRT*.

6.3. Experiment of Algorithm 3

Consider the special case presented in Sect. 5: obstacles satisfy the assumptions (3) and (4). In Figure 32-33, robot's vision is $r = 100$, start point $s = (140, 70)$ and goal point $g = (350, 350)$. Algorithm 3 gives the red dot polyline in Figure 33. The vertices of the red dot path are locations that the robot goes through. The red dot line segments are only pseudo path. The path that the robot goes through two such vertices is given by (33).

6.4. Experimental Works with Real Robots

We also applied successfully our approach to solve (BAR) problem to three real robots (TurtleBot3 burger, four wheeled robot, and PuppyPi Hiwonder Quadruped Robot, which have the vision ranges of 80mm, 100mm, 100mm, respectively in real environment) as shown in videos `https://youtu.be/CZDayeNFa6Q` , `https:`

//www.youtube.com/watch?v=DdPtubqQ54Y , and https://youtu.be/hz12ZOqgOa4 and shown in Figures 34 – 38. The source code of the integration our algorithm on these robots is given at https://drive.google.com/drive/u/1/folders/11j3lngyrkfiL-3ZVico-_rfz8XTHQQOU. During traveling of the robots, suitable sequences of bundles of line segments of the explored map are constructed for finding the shortest paths between two locations.

Consider the general case presented in Sect. 4. By (5) of Sect. 5, the higher the ranking $(x, y) \in \mathcal{M}(c)$ is, the more priority it is. Hence, our robot applies the formula

$$ranking(OP^l_{x,y}) \quad := \quad \text{sign}_{(x,y)}\, \mathcal{M}(c) \tag{7}$$

to score local active open point at (x, y), where $\text{sign}_{(x,y)}\, \mathcal{M}(c) = 1$ if $(x, y) \in \mathcal{M}(c)$, otherwise, $\text{sign}_{(x,y)}\, \mathcal{M}(c) = 0$. Hence, (7) can be seen as a special case of (1).

In Figure 39a, robot's vision is 100, start point $s = (10, 10)$ and goal point $g = (30, 70)$. Algorithms 1-2 give the red polyline in Figure 39.

To further evaluate the efficiency of our proposal in terms of travel path cost and number of turns, we also consider a scenario where some obstacles are placed randomly into blind alley region 1 (see Fig. 40). The outcome given in Table 2 indicates that our proposal outperforms both A* and RRT* with respect to travel path costs and numbers of turns.

Table 2.: In case of 10 randomly polygonal obstacles, our proposal is better than A* and RRT* with respect to travel path costs and numbers of turns.

Blind alley region 1	**Computing time**	**Travel path cost**	**Number of turns**
Our improved ASP	0.3489	215.8567	46
A*	0.0474	220.3796	58
RRT*	0.0022	248.0033	68

7. Conclusion Remarks

A novel algorithm for the problem of a robot in 2D with a limited vision range finding a path to a goal to avoid unknown polygonal obstacles has been presented. In some special cases, our algorithm is convergent with respect to $\|.\|$. To solve (BAR) problem (Blind Alley Region problem) that occurs during robot's traveling, our algorithm constructs sequences of bundles of line segments. Then a path-planning method given in the algorithm provides an available path between 2 points in blind alley regions. Comparisons of total lengths, number of turns, number of feasible paths between our proposal and A*, RRT* [10], RRTX [31] have been done. They show that our proposal is better in many cases. We already integrated our algorithm in to some robots.

The (BAR) problem can be also solved approximately by the Method of Multiple Shooting [7] or the Method of Orienting Curves [2], [3], and [5]. Analyzing an algorithm from the perspective of information theory (Entropy and Information Gain (a.k.a. Mutual Information)) can be both insightful and powerful. In addition, the cross-fertilization of motion and path planning and machine learning solves the (BAR) problem is interesting. These will be the subjects of other papers.

Acknowledgment

This research is funded by Vietnam National Foundation for Science and Technology Development (NAFOSTED) under Grant Number 102.01-2023.48. The authors acknowledge Ho Chi Minh City University of Technology (HCMUT), VNU-HCM for supporting this study. They would like to thank the anonymous reviewers for valuable comments and suggestions which help us greatly improve the quality of their manuscript.

Conflict of Interest Statement

The authors have no conflicts of interest to declare.

References

[1] Alonso-Mora, J. and Montijano, E. and Schwager, M. and Rus, D., Distributed multi-robot formation control among obstacles: A geometric and optimization approach with consensus, *IEEE International Conference on Robotics and Automation* (ICRA), Stockholm, Sweden (2016), pp. 5356–5363.

[2] An, P. T., Method of orienting curves for determining the convex hull of a finite set of points in the plane, *Optimization*, (2010), **59**, 2, 175–179.

[3] An, P. T., Finding shortest paths in a sequence of triangles in 3D by the Method of Orienting Curves, *Optimization*, (2018), **67**, 159–177.

[4] An, P. T., Optimization Approaches for Computational Geometry, Publishing House for Science and Technology - Vietnam Academy of Science and Technology, Hanoi. (2017).

[5] An, P. T. and Phu, H. X., Finding exactly shortest paths through a sequence of adjacent triangles by the Method of Orienting Curves, *Journal of Global Optimization*, (2023), **85**, 037–1063.

[6] An, P. T. and Hoai, T.V., Incremental Convex Hull as an Orientation to Solving the Shortest Path Problem, *International Journal of Information and Electronics Engineering*, (2012), **2**, 5.

[7] An, P. T. and Hai, N. N. and Hoai, T. V., Direct multiple shooting method for solving approximate shortest path problems, *Journal of Computational and Applied Mathematics*, (2013), **244**, 15, 67–76.

[8] Feng, A. and Xie, Y. and Sun, Y. and Wang, X. and Jiang, B. and Xiao, J., Efficient autonomous exploration and mapping in unknown environments, *Sensors*, (2023) **23**, 4766.

[9] Fox, D. and Burgard, W. and Thrun, S., The dynamic window approach to collision avoidance, *IEEE Robotics Automation Magazine*, (1997), **4**, 1, 23–33.

[10] Karaman, S. and Frazzoli, E., Sampling-based algorithms for optimalmotion planning, *Int. J. Rob. Res.*, (2011), **30**, 846–894.

[11] Kamon, I and Rimon, E. and Rivlin, E., TangentBug: A Range-Sensor-Based Navigation Algorithm, *The International Journal of Robotics Research*, (1998), **17**, 9, 934–953.

[12] Kamon, I. and Rivlin, E., Sensory-based motion planning with global proofs, *IEEE Transactions on Robotics and Automation*, (1997), **13**, 6, 814–822.

[13] Kamil, F., and Moghrabiah, M. Y., Multilayer decision-based fuzzy logic model to navigate mobile robot in unknown dynamic environments. *Fuzzy Information and Engineering*, (2021) **14**(1), 51–73.

[14] Kang, X., Yue, Y., Li, D. and Maple, C., Genetic algorithm based solution to dead-end problems in robot navigation. *International Journal of Computer Applications in Technology*, (2011) **41**, 177–184.

[15] Kovács, B. and Szayer, G. and Tajti, F. and Burdelis, M. and Korondi, P. A novel potential field method for path planning of mobile robots by adapting animal motion attributes, *Robotics and Autonomous Systems*, (2016), **82**, 24–34.

[16] Kunchev, V. and Jain, L. and Ivancevic, V. and Finn, A., Path planning and obstacle avoidance for autonomous mobile robots: a review, *Int. Conf. Knowl. Based Intell. Inf. Eng. Syst.*, Springer, Berlin, Heidelberg. (2006), 537–544.

[17] Kuffner, J.J. and LaValle, S.M., RRT-connect: An efficient approach to single-query path planning, *Proceedings 2000 ICRA Millennium Conference. IEEE International Conference on Robotics and Automation.* Symposia Proceedings (Cat. No.00CH37065), (2000), **2**, 995–1001.

[18] Hai, N. N. and An, P. T. and Huyen, P. T. T., Shortest paths along a sequence of line segments in Euclidean spaces, *Journal of Convex Analysis*, (2019), **26**, 4, 1089–1112.

[19] Horiuchi, Y. and Noborio, H., Evaluation of path length made in sensor-based path-planning with the alternative following, *Proceedings 2001 ICRA. IEEE International Conference on Robotics and Automation* (Cat. No.01CH37164), (2001), **2**, 1728–1735.

[20] Hsu, D. and Latombe, J.-C. and Kurniawati, Hanna, On the Probabilistic Foundations of Probabilistic Roadmap Planning, Robotics Research, Springer, Berlin, Heidelberg, (2007), 83–97.

[21] Ladd, A.M. and Kavraki, L.E., Measure theoretic analysis of probabilistic path planning, *IEEE Transactions on Robotics and Automation*, (2004), **20**, 2, 229–242.

[22] Lavalle, M. S., Rapidly-Exploring Random Trees: A New Tool for Path Planning, (1998) TR 98-11, Computer Science Dept., Iowa State University.

[23] Ladd, A.M. and Kavraki, L.E., Measure theoretic analysis of probabilistic path planning, *IEEE Transactions on Robotics and Automation*, (2004), **20**, 2, 229–242.

[24] Li, F. and Klette, R., Euclidean Shortest Paths: Exact or Approximate Algorithms, Springer, London. (2011).

[25] Li, F. and Klette, R., Rubberband Algorithms, Euclidean Shortest Paths: Exact or Approximate Algorithms, Springer, London, (2011), 53–89.

[26] Lumelsky, V. J. and Stepanov, A. A., Path-planning strategies for a point mobile automaton moving amidst unknown obstacles of arbitrary shape, *Algorithmica*, (1987), **2**, 1-4, 403–430.

[27] Lumelsky, V.J. and Skewis, T., Incorporating range sensing in the robot navigation function, *IEEE Transactions on Systems, Man, and Cybernetics*, (1990), **20**, 5, 1058–1069.

[28] Manikandan, N.S. and Kaliyaperumal, G., Collision avoidance approaches for autonomous mobile robots to tackle the problem of pedestrians roaming on campus road, *Pattern Recognition Letters*, (2022), **160**, 112–121.

[29] Ng, J. and Braunl, T., Performance comparison of bug navigation algorithms, *Journal of Intelligent and Robotic Systems*, (2007), **50**, 73–84.

[30] A. H. Qureshi, A. Simeonov, M. J. Bency and M. C. Yip, Motion planning networks, *2019 International Conference on Robotics and Automation (ICRA)*, Montreal, QC, Canada, 2019, pp. 2118–2124.

[31] Otte, M. an d Frazzoli, E., RRTX: Asymptotically optimal single-query sampling-based motion planning with quick replanning, *The International Journal of Robotics Research*, (2016), **35**, 7, 797–822.

[32] Roy, D. and Maitra, M. and Bhattacharya, S., Exploration of multiple unknown areas by swarm of robots utilizing virtual-region-based splitting and merging technique, *IEEE Transactions on Automation Science and Engineering*, (2022) **19**, 4, 3459–3470.

[33] Rosell, J. and Iniguez, P., Path planning using Harmonic functions and probabilistic cell decomposition, *Proceedings of the 2005 IEEE International Conference on Robotics and Automation*, (2005), 1803–1808.

[34] Seda, M., Roadmap Methods vs. Cell Decomposition in Robot Motion Planning, *Proceedings of the 6th WSEAS International Conference on Signal Processing, Robotics and Automation*, World Scientific and Engineering Academy and Society (WSEAS), (2007), **6**, 127—132.

[35] Sankaranarayanan, A. and Vidyasagar, M, A new path planning algorithm for moving a point object amidst unknown obstacles in a plane, *Proceedings of IEEE International Conference on Robotics and Automation*, (1990), **3**, 1930–1936.

[36] Sankaranarayanar, A. and Vidyasagar, M., Path planning for moving a point object amidst unknown obstacles in a plane: a new algorithm and a general theory for algorithm development, *29th IEEE Conference on Decision and Control*, (1990), **2**, 1111–1119.

[37] Tashtoush, Y., Haj-Mahmoud, I., Darwish, O., Maabreh, M., Alsinglawi, B., Elkhodr, M. and Alsaedi, N. (2021). *Enhancing Robots Navigation in Internet of Things Indoor Systems*. Computers, 10(11), 153.

[38] A. -I. Toma et al., Waypoint planning networks, *18th Conference on Robots and Vision (CRV)*, Burnaby, BC, Canada, 2021, pp. 87–94,

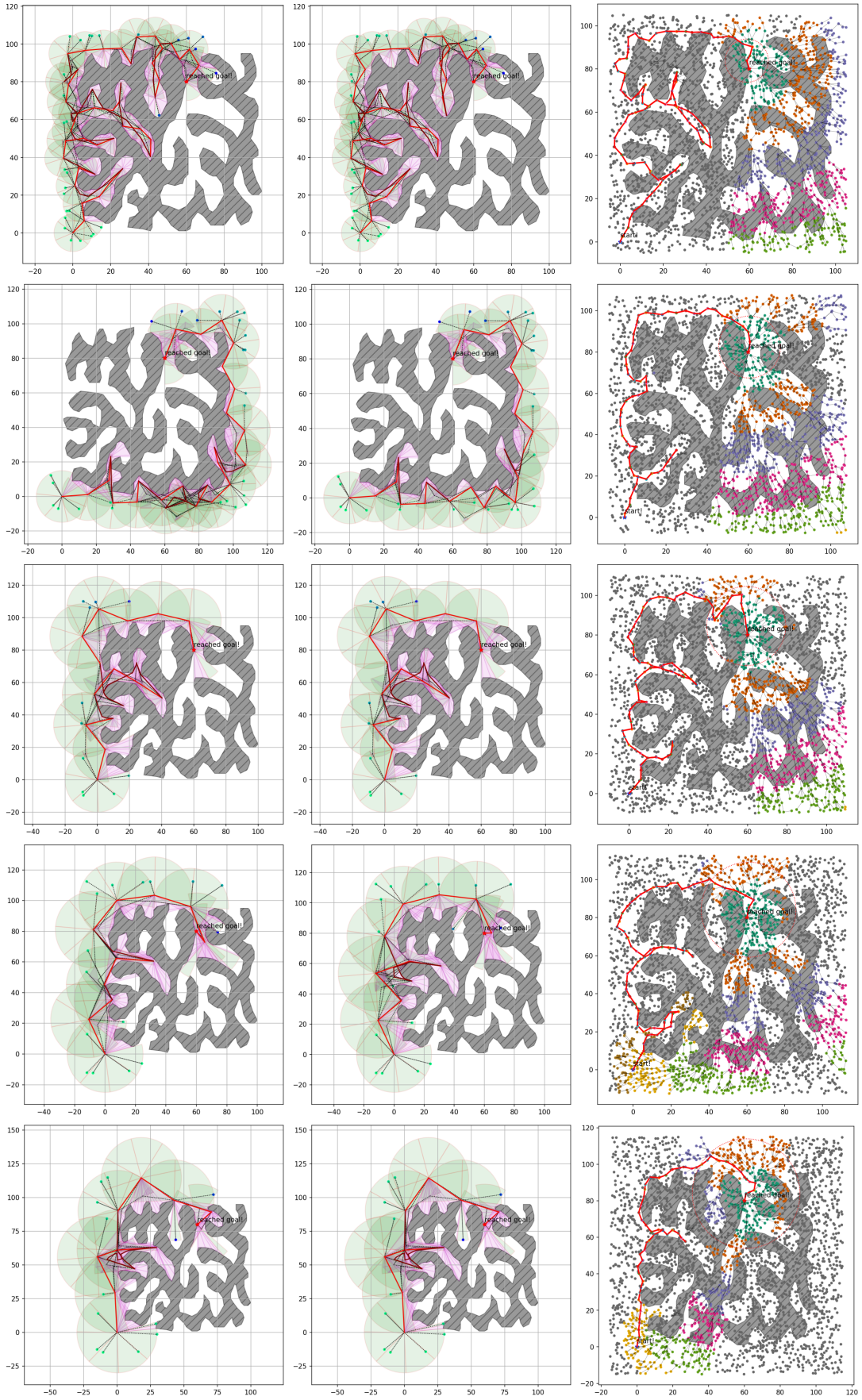


Figure 24.: Composite image of generated trajectory of among [our improved] picking global set first, [our improved] picking neighbor set first, and RRTX (from left to right); start s =(0, 0), goal g =(60, 80) in blind alley region 1.

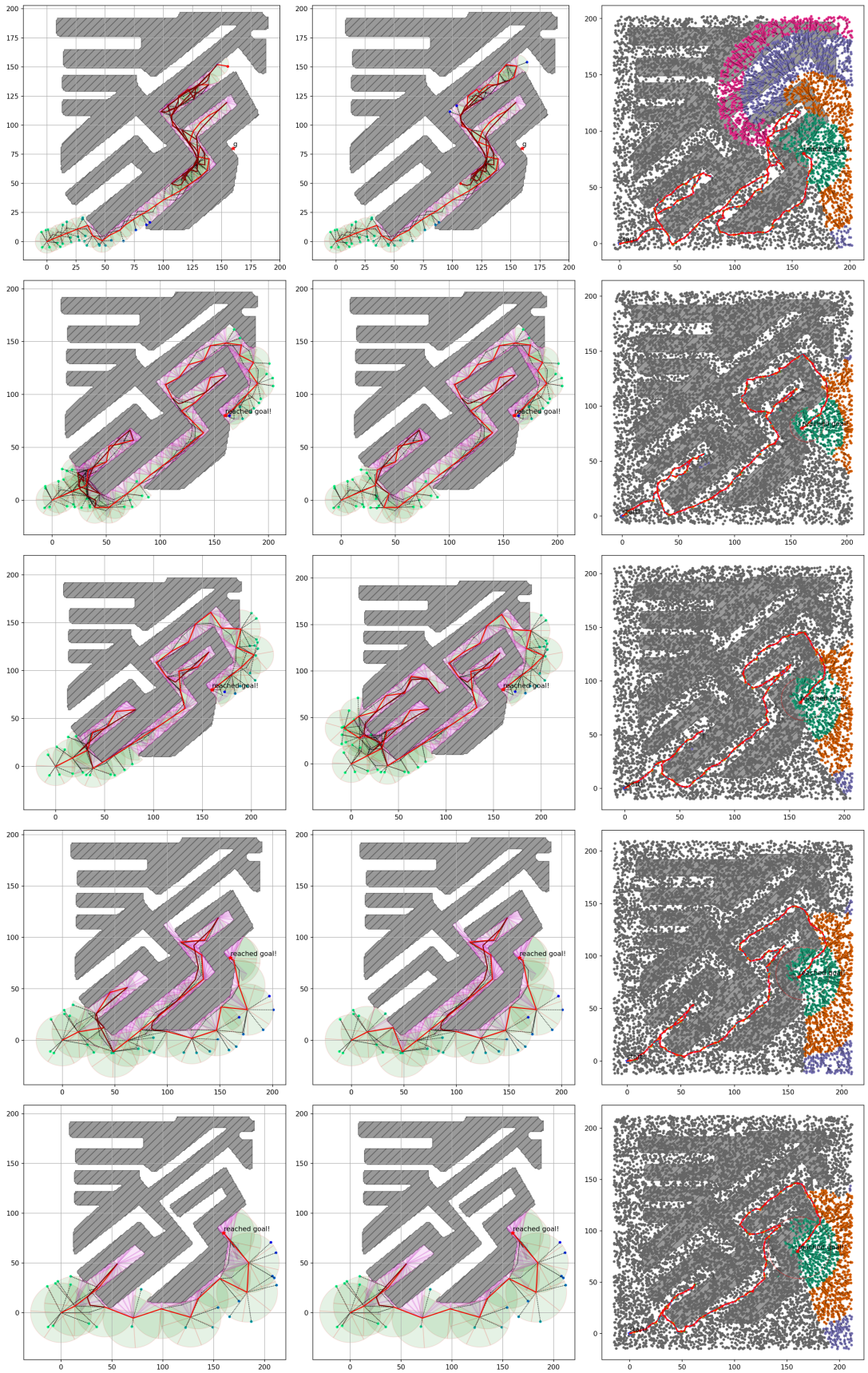


Figure 25.: Composite image of generated trajectory of among [our improved] picking global set first, [our improved] picking neighbor set first, and RRTX (from left to right); start s =(0, 0), goal g =(160, 80) in blind alley region 2

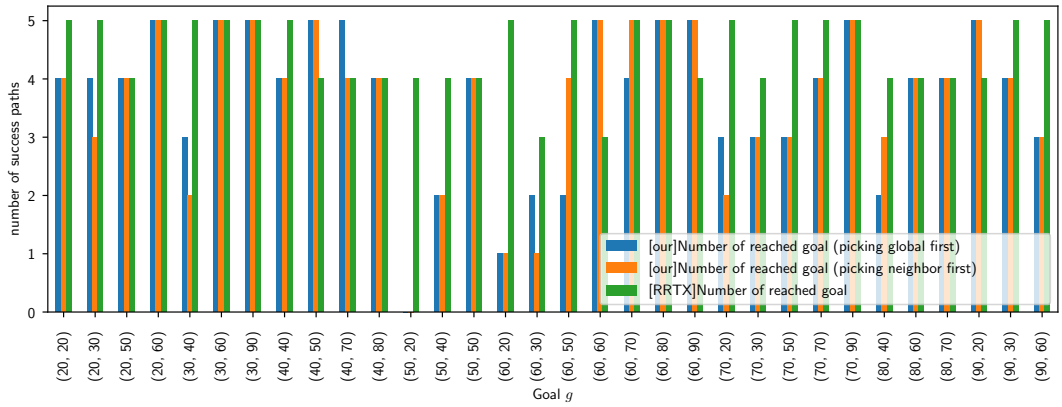


Figure 26.: Comparison number of feasible paths between our proposal and RRTX on blind alley region 1

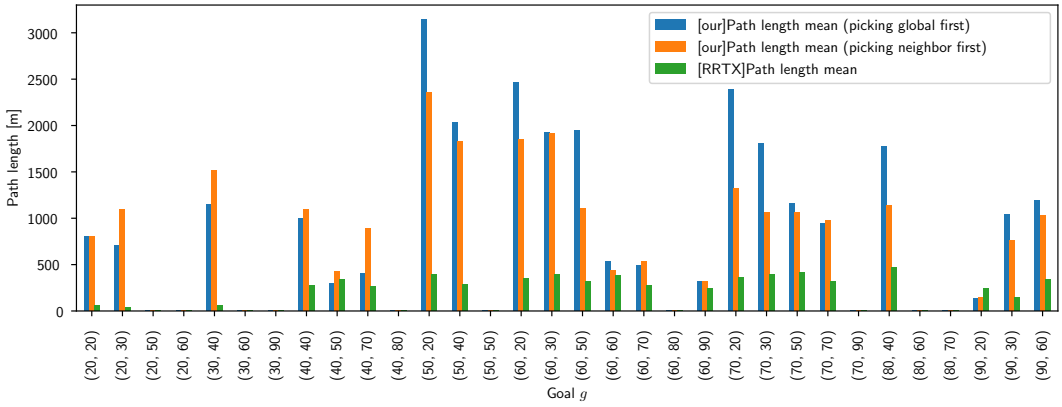


Figure 27.: Comparison number of total path lengths between our proposal and RRTX on blind alley region 1.

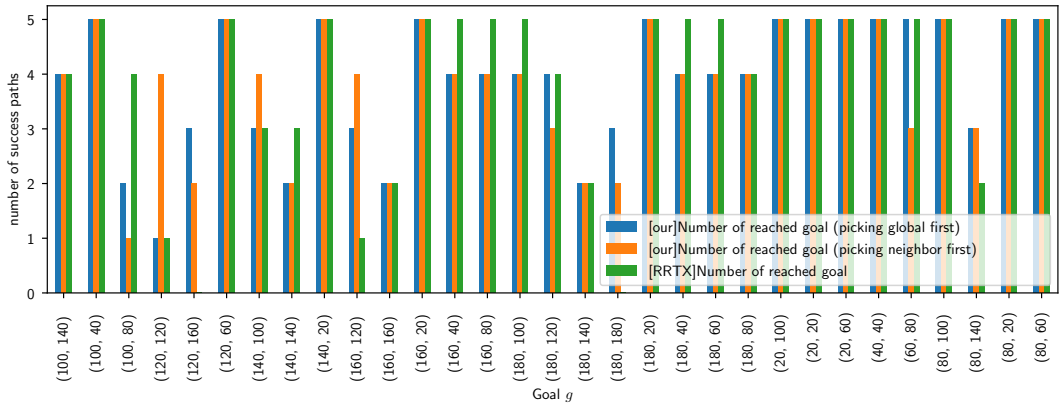


Figure 28.: Comparison number of feasible paths between our proposal and RRTX on blind alley region 2. That number of RRTX is lightly bigger than our one.

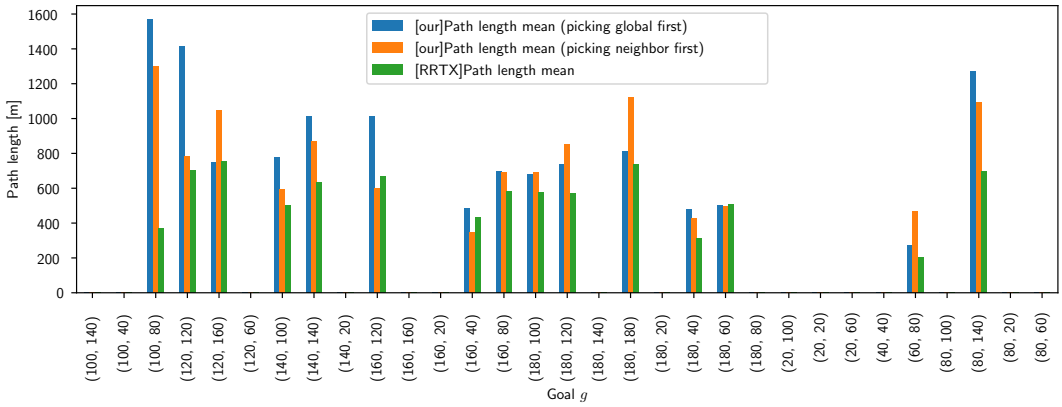


Figure 29.: Comparison number of total path lengths between our proposal and RRTX on blind alley region 2.

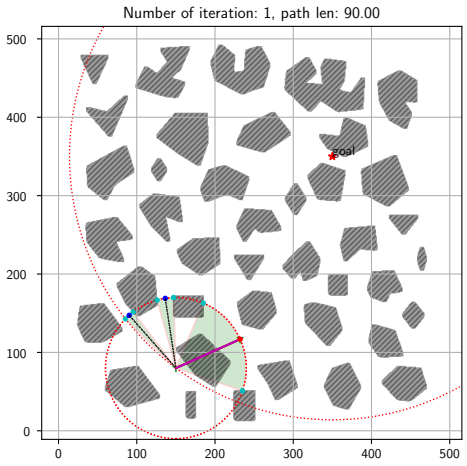


Figure 30.: The set $\mathcal{M}(c)$ at $c = (150, 80)$ and the goal $g = (350, 350)$.

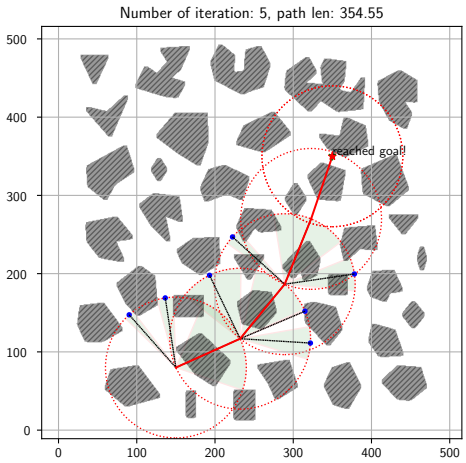


Figure 31.: Algorithm 3: The red path created by the robot joining the start point and the goal.

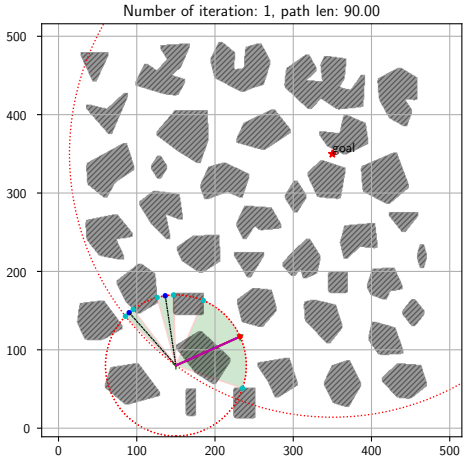


Figure 32.: The set $\mathcal{M}(c)$ at $c = (150, 80)$ and the goal $g = (350, 350)$.

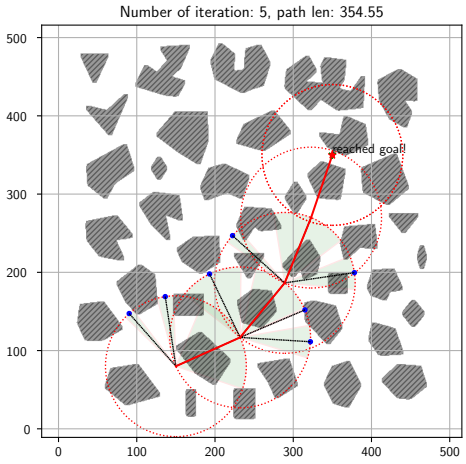


Figure 33.: Algorithm 3: The red path created by the robot joining the start point and the goal.

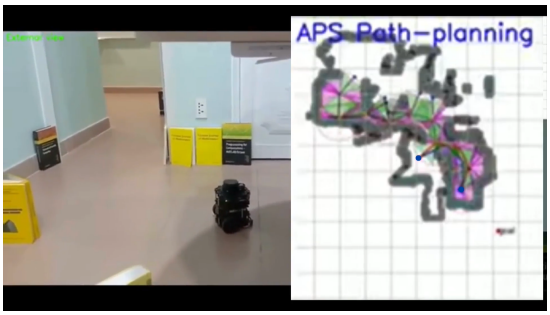


Figure 34.: Turtle-bot3 burger with the vision range of $r = 80mm$ in the real world. Sequences of bundles of line segments are displayed on the right.

Figure 35.: Turtle-bot3 burger's blue path (returned path to escape the blind alley region) is shorter than the red one.

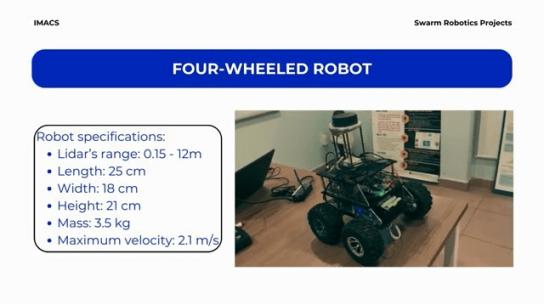


Figure 36.: The four-wheeled robot with the vision range of $r = 100mm$.

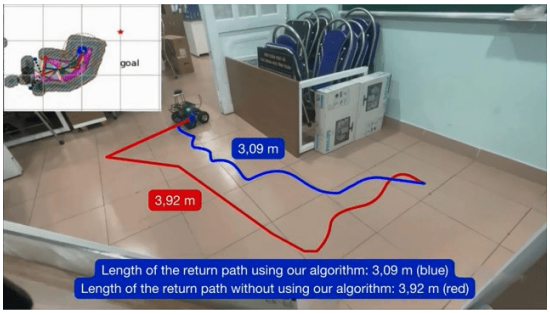


Figure 37.: The returned path coloured in blue determined by our algorithm for the four-wheeled robot on the sequence of bundle of line segments is shorter the red one. Such a sequence is displayed on the windows in the upper left corner.

Figure 38.: PuppyPi Hiwonder Quadruped Robot with $r = 100mm$. The returned path coloured in blue determined by our algorithm on the sequence of bundle of line segments is shorter the red one.

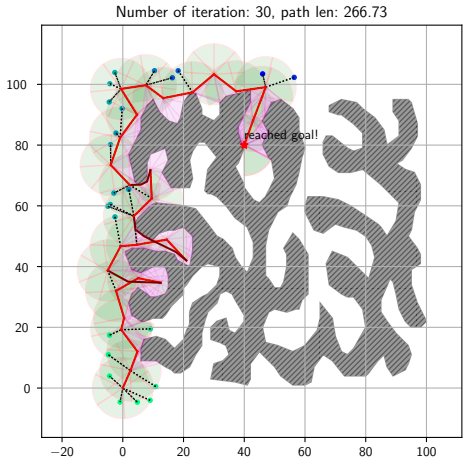


Figure 39.: Algorithms 1-2: The red and dark red path created by the robot joining the start point and the goal.

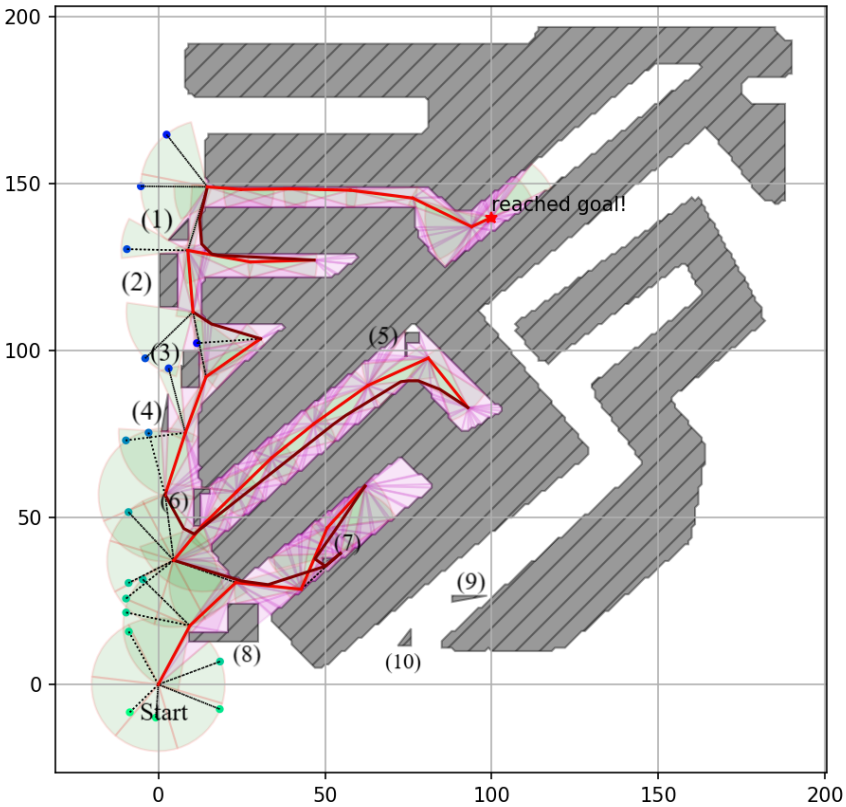


Figure 40.: Traveled paths created by our Algorithm 2 on blind alley region 1 where 10 polygonal obstacles are given randomly (they are numbered from (1) to (10).